CHAPTER-1

INTRODUCTION

In today's digital era, the workplace is no longer confined to a physical desk within office walls. Employees carry their work in their pockets—smartphones, tablets, laptops—and access corporate resources from coffee shops, homes, and even while travelling. This mobility has unlocked immense productivity and flexibility, yet it has also opened doors to a new class of security challenges. Every device that connects to corporate data becomes a potential entry point for attackers. And as organizations increasingly adopt cloud services and bring-your-own-device policies, the line between personal and professional use blurs, making security enforcement both critical and delicate.

Imagine this: you are an employee at a leading aerospace company. You step into the office each morning, badge in hand, ready to begin your day. But what happens to the mobile device in your pocket? Should it have the same access inside the secure facility as it does when you are at home? If a camera is allowed in the cafeteria, is it also allowed in the design lab? These are the kinds of questions that modern security teams grapple with daily. The traditional approach—manually configuring restrictions on each device or applying blanket policies—is slow, error-prone, and often frustrating for users who feel their personal tools are being unnecessarily constrained.

I experienced this challenge firsthand during my internship at Tata Advanced Systems Limited (TASL), a premier aerospace and defence company in India. Working alongside the IT security team, I saw the constant struggle to enforce security without disrupting the workflow of engineers, managers, and other staff. Devices would be enrolled in management systems, but policy updates often lagged behind physical presence. An employee could walk into a sensitive area with a fully functional camera on their phone, because the security policy hadn't yet "caught up." Meanwhile, users grumbled about having to repeatedly authenticate or losing access to personal apps they needed during breaks.

This project grew from that observation: what if we could make the security system aware of where an employee actually is? What if the moment you badge in, your device automatically enters a "restricted" mode—camera off, personal apps blocked—and the moment you leave, full access is restored—without a single click from IT? And more than that, what if the system could explain why these changes happen, giving users clarity and trust rather than confusion?

That is the idea behind this research. We set out to build a bridge between the physical world of access control (like the WinPak system that tracks employee entry and exit) and the digital world of device management (Microsoft Intune). By linking these two systems through an intelligent API-driven workflow, we created an automated solution that responds to real-world events in real time. But we didn't stop at technical implementation. We also wanted to understand how these security features affect the people who use them every day. Does requiring multi-factor authentication multiple times a day truly make us safer, or does it just make us more frustrated? Are there ways to balance protection with productivity?

This report tells the story of that journey—from learning the intricacies of Intune and WinPak, to designing and testing an integrated solution, to measuring the real-world impact on users across 650 devices in multiple organizations. It is a story about security, yes, but also about transparency, usability, and the human element that too often gets overlooked in technical projects. Because in the end, the best security is the one that people understand and accept—not the one they try to bypass.

1.1 Background

The modern enterprise has undergone a quiet revolution. A decade ago, most work was done on company-issued desktops, behind firewalls, inside buildings. Today, employees carry their work on personal smartphones, log in from home networks, and share files across cloud platforms. This shift has brought tremendous benefits—employees can collaborate across continents, respond to urgent matters from anywhere, and enjoy a level of flexibility that boosts morale and retention.

But this flexibility comes at a cost. Every new device adds a potential vulnerability. According to industry reports, over 70% of enterprises now support bring-your-own-device (BYOD) policies, yet mobile-related security incidents have risen sharply in recent years (IBM Security, 2025). Attackers increasingly target mobile endpoints because they are often less protected than corporate laptops and because they hold a wealth of sensitive data—emails, contacts, documents, even biometric information.

At the heart of the problem is a fundamental tension: security measures, by their nature, introduce friction. A system that requires a password, a fingerprint, and a one-time code for every login is undeniably more secure than one that does not, but it also slows down the user. If the friction

becomes too great, users will find workarounds—sharing passwords, disabling security features, or simply refusing to use corporate apps on their devices. This "shadow IT" can actually make the organization less secure than if it had adopted a moderate, well-explained set of controls.

Microsoft Intune was designed to address these challenges. As a cloud-based enterprise mobility management (EMM) solution, it provides tools to manage devices, applications, and data across platforms, all from a single console. It can enforce encryption, require compliance, push apps, and even wipe devices remotely. But like any powerful tool, its effectiveness depends on how it is configured—and whether users understand and accept the policies it enforces.

At TASL, the need for such a solution was acute. The company handles sensitive aerospace and defence projects, where intellectual property protection is not just a business requirement but a national security imperative. Yet the workforce includes engineers who need to collaborate with partners, managers who travel frequently, and administrative staff who rely on mobile access for day-to-day operations. Balancing the strict security demands of the industry with the practical needs of employees required a solution that was both intelligent and transparent.

That is why we turned to an integration approach: linking the physical access control system (WinPak) with Intune. If we could automatically adjust device policies based on whether an employee was inside the secure facility, we could enforce stronger controls exactly when and where they were needed, and relax them when they were not. This would reduce friction for employees working remotely or in non-sensitive areas, while ensuring that the highest level of protection was applied automatically when entering restricted zones.

1.2 Statistics

The scale of the problem is reflected in hard numbers. According to the IBM Security Cost of a Data Breach Report 2025, the average cost of a data breach involving mobile devices has climbed significantly in recent years, with credential theft—often facilitated by phishing attacks targeting mobile users—remaining the leading cause (IBM Security, 2025). In India, the Ministry of Electronics and Information Technology reported a steady increase in mobile malware incidents, driven by the rapid digitization of government services and enterprise operations (MeitY, 2024). More than 60% of Indian enterprises now have a formal BYOD policy, but many have not yet implemented comprehensive mobile threat defense solutions, leaving a significant gap in protection (Gartner, 2025).

At TASL, internal audits revealed that prior to this project, policy enforcement delays averaged over 30 minutes—meaning an employee could be inside a secure area for half an hour before their device restrictions took effect. Manual configuration errors occurred in approximately 12% of device enrollments, leading to either overly permissive or overly restrictive settings. These statistics underscored the urgent need for an automated, reliable solution.

1.3 Proposed approach

Technology Learning: The initial weeks were dedicated to studying Microsoft Intune: its architecture, policy types, enrollment workflows, and integration capabilities. We also explored WinPak, the physical access system used at TASL, to understand how it generates events and exposes APIs. Finally, we delved into the Microsoft Graph API, which would serve as the bridge between these two worlds, and the Azure cloud services that would host our automation logic.

System Design and Implementation: We designed and built a proof-of-concept integration. We created an Azure Function that listens for punch-in and punch-out events from WinPak. Upon receiving an event, the function authenticates to the Graph API and moves the employee's device into the appropriate Intune group ("Restricted" for punch-in, "Unrestricted" for punch-out). This group membership triggers pre-configured policies, such as disabling the camera or blocking personal apps. The entire workflow was designed to be fully automated, with a target policy application time of under five minutes.

Testing and Validation: The system was deployed in a controlled environment first, then gradually expanded to production devices at TASL. We collected data on policy application latency, error rates, and device compliance to validate the reliability of the integration.

Documentation and Best Practices: We documented the entire process, including configuration steps, troubleshooting tips, and recommendations for organizations seeking to implement similar solutions. The final deliverable includes a comprehensive guide for deploying Intune with WinPak integration.

1.4 Objectives

The following objectives were defined to guide the project work:

Objective 1: Gain comprehensive understanding of Microsoft Intune, WinPak, Graph API, and Azure cloud services.

This involves studying the architecture, capabilities, and configuration of Intune for both iOS and Android devices, understanding the physical access control system (WinPak) and its event-triggering mechanisms, and mastering the use of Microsoft Graph API and Azure Functions to enable automated workflows.

Objective 2: Design and implement an automated integration between WinPak and Microsoft Intune.

The goal is to create a seamless, event-driven system where employee badge-in and badge-out events from WinPak automatically move devices into appropriate Intune groups, triggering pre-defined security policies (e.g., disabling camera, restricting app access) without any manual intervention.

Objective 3: Validate the performance and reliability of the integrated solution. This includes testing the system in a controlled laboratory environment, then deploying it to a limited set of production devices at Tata Advanced Systems Limited (TASL). Key metrics to be measured are policy application latency, success rate of device group updates, and overall system stability.

Objective 4: Evaluate the impact of automated policy enforcement on IT operations and user experience.

Through monitoring and feedback collection, assess how the automated solution reduces manual IT effort, minimizes errors, and affects end-user satisfaction. Document any challenges encountered and identify areas for improvement.

Objective 5: Create comprehensive documentation and best practice guidelines. Develop a detailed implementation guide covering the architecture, configuration steps, troubleshooting tips, and operational procedures. This documentation will enable other organizations to replicate the solution and adopt similar context-aware security practices.

1.5 Overview of project report

This project report provides a comprehensive account of the work undertaken to integrate Microsoft Intune with the WinPak physical access control system at Tata Advanced Systems Limited (TASL), and to evaluate the resulting automated security workflow. The report guides the reader through the technical decisions, implementation steps, and outcomes, following a structured format.

Chapter 1 sets the stage by introducing the modern workplace challenges that necessitate intelligent mobile device management. It discusses the tension between security and user experience, presents relevant statistics, reviews prior technologies, and outlines the proposed approach, objectives, and alignment with the Sustainable Development Goals.

Chapter 2 provides a detailed literature review, summarizing key academic and industry publications on mobile device management, enterprise mobility, and integration with physical access systems. This chapter situates the project within the broader research landscape and identifies the gaps that this work addresses.

Chapter 3 describes the methodology adopted for the project. Using the V-Model approach, it outlines the phases from requirements analysis through system design, implementation, testing, and validation. This chapter explains how the project was structured to ensure quality and traceability.

Chapter 4 addresses project management aspects, including the detailed project timeline (Gantt chart), risk analysis using a PESTLE framework, and project budget. It reflects how the work was planned and executed within the constraints of time, resources, and organizational priorities.

Chapter 5 presents the analysis and design of the system. It covers the requirements gathering, functional block diagram, system flow chart, device selection, standards compliance, and mapping to the IoT World Forum Reference Model. This chapter lays the technical foundation for the implementation.

Chapter 6 details the practical implementation process, including the hardware used (test devices), software development tools (Visual Studio Code, PowerShell, Postman), and the core code of the Azure Function that ties WinPak events to Intune group membership. It also covers simulation techniques used to validate the integration.

Chapter 7 outlines the evaluation and results. This includes the test points defined, the test plan, and the outcomes of the implementation. Metrics such as policy application latency, success rates, and system reliability are presented, along with insights gained from the testing phase.

Chapter 8 explores the broader implications of the project, including social impacts (user acceptance, trust), legal compliance (data privacy regulations), ethical considerations (balancing security with autonomy), sustainability (cloud resource efficiency), and safety aspects (protection against cyber threats).

Chapter 9 concludes the report by summarizing the key achievements, reflecting on the challenges encountered, and suggesting future directions for enhancing the solution, such as incorporating AI for adaptive policies or extending integration to other physical access systems.

This systematic structure ensures that the reader can follow the entire journey—from problem identification and design to implementation, testing, and reflection—providing both a technical blueprint and a strategic guide for similar initiatives in enterprise mobility management.

Chapter 2

LITERATURE REVIEW

2.1 Foundational Research in Mobile Device Management

The study by De los Reyes et al. (2012) proposed a security architecture for mobile enterprises using network-based security and cloud computing. Their work laid the foundation for cloud-centric MDM solutions, emphasizing the need for centralized policy enforcement across diverse device types. The researchers identified three core challenges in mobile enterprise security: device heterogeneity, network diversity, and the constant threat of data leakage. Their proposed architecture introduced the concept of separating management functions from on-premises infrastructure, moving them to the cloud to enable scalability and remote administration. While innovative for its time, the study did not address the usability implications of stringent security measures, nor did it explore how policy enforcement might be tied to physical presence or real-time events. The researchers noted that successful enterprise mobility solutions must evolve to handle the increasing complexity of mobile ecosystems, setting the stage for later developments in cloud-based MDM platforms.

2.2 Cloud-Based MDM Architectures

Song and Yang (2017) developed a comprehensive mobile device management system based on the Android Open Source Project (AOSP) and SELinux. Their system offered real-time location tracking, file verification, permission control, and remote wipe capabilities. The researchers demonstrated the feasibility of deep OS-level integration for security enforcement, showing that customized Android builds could provide enhanced security compared to standard deployments. Their implementation included a centralized management console that allowed administrators to monitor device compliance, push configuration updates, and respond to security incidents. However, the study revealed significant limitations: customizing AOSP for each device model was resource-intensive, updates required manual intervention, and scalability across large device fleets was problematic. The researchers concluded that while OS-level integration offers superior security, the maintenance overhead makes it impractical for heterogeneous enterprise environments with diverse device models and operating systems.

2.3 Enterprise Information Systems and Mobility

Baltazar et al. (2020) explored how Enterprise Information Systems (EIS) can drive improved and sustainable mobility, using a carsharing platform case study to illustrate practical applications. Their research highlighted the role of artificial intelligence and machine learning in security, suggesting that predictive analytics could enhance threat detection and automate incident response. The study emphasized that successful mobility solutions must balance technical capabilities with business process integration, ensuring that security measures do not disrupt operational workflows. The researchers also noted the growing importance of real-time data processing in enterprise mobility, where delayed policy updates could leave security gaps. While their work focused on a specific industry application, the principles of integrating security with business processes informed the design of automated, event-driven MDM solutions.

2.4 Framework for Securing Mobile Enterprise Applications

Hasan et al. (2014) presented a comprehensive framework for securing mobile enterprise applications, including a meta-model (UML diagram) describing framework components, a guidance model listing mobile security threats and countermeasures, and decision-making processes for logging security design decisions. Their framework supported enterprises in systematically addressing security requirements during application development. The researchers identified key threat categories: data leakage, unauthorized access, malware injection, and man-in-the-middle attacks. They proposed countermeasures including encryption, certificate pinning, secure storage, and runtime application self-protection. However, the framework's static nature made it less effective against new threats and zero-day attacks that had not been previously documented. The researchers acknowledged that security frameworks must be continuously updated to remain effective, highlighting the need for adaptive, learning-based security mechanisms.

2.5 Comparative Analysis of MDM Vendors

Batool and Masood (2020) conducted a comprehensive comparative analysis of leading MDM solutions, including AirWatch by VMware, IBM MaaS360, and Kaspersky Lab MDM. Their study used a yes/no checklist to evaluate security requirements such as profile management, location tracking, remote lock and wipe, malware detection, data backup and encryption, and URL blacklisting. The researchers found that while all major vendors satisfied basic security requirements, significant differences existed in deployment complexity, platform support, and integration capabilities. AirWatch excelled in comprehensive feature sets but required extensive configuration; IBM MaaS360 offered strong reporting capabilities but limited customization;

Kaspersky provided robust malware protection but had fewer integration options. The study revealed that vendor selection involves trade-offs between feature completeness, ease of management, and cost. However, the researchers acknowledged that feature comparison alone does not measure real-world effectiveness or user experience impact, a gap their study did not address.

2.6 Enterprise Mobility Strategy Evaluation

Yucel (2017) proposed a generic model for evaluating enterprise mobility strategies based on parameters such as objectives, users, mobile solutions, mobile and backend platforms, benefits, drawbacks, risks and mitigation plans, strategies for distribution, launch, marketing, promotion and positioning, and costs. The researcher developed a model that counts the rate of potential users and shows relationships between factors such as user adoption, benefits, and economic utilities. The model aimed to increase return on investment while receiving benefits from enterprise mobility. However, the study remained theoretical, offering limited practical guidance for technical implementation. The researcher noted that successful mobility strategies require alignment between business goals and technical capabilities, emphasizing the need for interdisciplinary approaches that consider both organizational and technical factors.

2.7 MDM Technologies, Protocols, and Challenges

Yamin and Katt (2019) conducted an extensive review of mobile device management technologies, including enabling protocols such as Smart Message, Open Mobile Alliance Device Management (OMA-DM), and Over-The-Air Programming (OTAP). Their research examined the evolution of MDM from simple device tracking to comprehensive security management encompassing application control, content management, and identity protection. The researchers identified persistent challenges: device fragmentation across different manufacturers and OS versions, user privacy concerns when implementing location tracking and monitoring, interoperability issues between management platforms, and the inherent complexity of managing diverse device types. Their analysis revealed that while MDM protocols have matured significantly, practical implementation remains challenging due to the rapid pace of mobile technology evolution and the increasing sophistication of security threats.

2.8 Mobile Application Management and Application Pushing

Suduca and Bizoia (2022) explored ways of integrating and pushing applications through Microsoft Intune, demonstrating how the platform minimized application pushing effort using

the videoconferencing application Zoom as a case study. Their research showed that leveraging Intune's built-in application deployment capabilities significantly reduced the time required to distribute and manage applications across large device fleets. The researchers highlighted that automated app deployment reduces manual effort, ensures consistent configuration, and enables rapid security updates. However, their study did not explore the broader security implications of application management, such as the balance between automated deployment and user choice, or the impact of mandatory applications on user experience. The researchers suggested that future work should examine how application management policies affect user satisfaction and productivity.

2.9 Identity Protection and BYOD Policies

Medudula et al. (2016) discussed the challenges of identity protection in the context of BYOD policies, emphasizing the need for secure mobile application management and strong authentication mechanisms. Their work explored how traditional perimeter-based security models break down when employees use personal devices to access corporate resources. The researchers proposed a layered security approach combining device-level controls, application-level protections, and identity management. They noted that BYOD environments require careful balancing of security requirements with employee privacy expectations, suggesting that organizations should develop clear policies that specify what data is collected, how it is used, and what controls are applied to personal devices. The study highlighted that successful BYOD programs depend on transparent communication and user education to build trust and ensure compliance.

2.10 Security Architecture for Mobile Enterprises

In foundational work that influenced modern MDM approaches, De los Reyes et al. (2012) proposed a security architecture for mobile enterprises that combined network-based security with cloud computing. Their architecture separated management functions from device operations, enabling centralized policy enforcement without requiring constant on-premises connectivity. The researchers identified key components: device registration and authentication, policy distribution, compliance monitoring, and incident response. Their work anticipated the shift toward cloud-based management platforms that could scale to support large, geographically distributed workforces. However, the architecture was designed before the widespread adoption of BYOD policies and did not address the unique challenges of managing personal devices, nor did it consider the user experience implications of security controls.

2.11 Summary of Literature Reviewed

Table 2.1 summarizes the main findings, methods and limitations of the literature.

Table 2.1: Summary of Literature Reviews

Author & Year	Key Contribution	Methodology	Limitations	Relevance to Our Work
De los Reyes et al. (2012)	Cloud-based security architecture for mobile enterprises	Architectural design and analysis	No usability analysis; pre-BYOD context	Foundation for cloud-centric MDM approaches
Song & Yang (2017)	AOSP-based MDM with SELinux integration	System implementation and testing	Scalability issues; maintenance overhead	Demonstrated OS-level security integration
Baltazar et al. (2020)	AI/ML role in enterprise mobility	Case study analysis	Not MDM-specific; single industry focus	Highlighted predictive analytics potential
Hasan et al. (2014)	Mobile app security framework	Meta-model and guidance development	Static framework; zero-day gaps	Informed app protection policy design
Batool & Masood (2020)	MDM vendor comparison	Feature-based checklist analysis	No effectiveness metrics; no user experience	Validated Intune's comprehensive capabilities
Yucel (2017)	Mobility strategy evaluation	Theoretical model development	No technical implementation guidance	Informed project management approach

Yamin & Katt (2019)	MDM technologies and protocols	Literature review and analysis	No product-specific evaluation	Provided protocol and architecture context
Suduca & Bizoia (2022)	Intune application deployment	Practical implementation case study	No security trade-off analysis	Demonstrated Intune's automation potential
Medudula et al. (2016)	BYOD identity protection	Policy and framework discussion	No quantitative data	Informed BYOD security requirements

2.12 Identified Research Gaps and Challenges

Analysis of the literature reveals several significant gaps that this project addresses:

Gap 1: Integration of Physical Access Systems with MDM

Existing literature focuses almost exclusively on digital security controls, with no studies examining the integration of physical access systems (such as badge-based entry) with mobile device management. This gap is particularly significant in security-sensitive industries where physical presence correlates with data sensitivity. Our project addresses this by developing and documenting a working integration between WinPak and Microsoft Intune.

Gap 2: Quantification of User Experience Impact

The majority of existing research evaluates security features based on technical capabilities rather than their real-world impact on users. While several studies acknowledge the tension between security and usability, none provide quantitative frameworks for measuring or predicting user experience impact. Our project fills this gap by collecting and analyzing user feedback on specific security features, providing statistical insights that can guide implementation decisions.

Gap 3: Real-Time, Event-Driven Policy Enforcement

Most MDM deployments rely on periodic policy updates or manual interventions, creating delays between security-relevant events and policy changes. The literature does not adequately address real-time, event-driven enforcement triggered by physical presence or other contextual factors. Our project demonstrates how API-based integration enables immediate policy updates in response to real-world events.

Gap 4: Transparent and Explainable Security Controls

Many MDM solutions operate as black boxes, applying policies without providing users with clear explanations of why certain restrictions are in place. This lack of transparency can lead to user frustration and attempts to bypass controls. Our project emphasizes transparency by clearly linking policy changes to physical events, helping users understand the logic behind security measures.

Gap 5: BYOD-Specific Implementation Guidelines

While BYOD policies are widely adopted, the literature offers limited practical guidance on configuring MDM solutions to balance corporate security requirements with user privacy expectations on personal devices. Our project provides specific configuration recommendations for BYOD environments, documenting which controls are appropriate for personal devices and which should be reserved for corporate-owned equipment.

Gap 6: Performance Metrics for Automated MDM Workflows

Existing studies lack standardized metrics for evaluating automated MDM workflows, making it difficult to compare solutions or establish performance benchmarks. Our project defines and measures key metrics such as policy application latency, success rates, and system reliability, providing a framework for evaluating automated MDM implementations.

These gaps represent opportunities for meaningful contribution to the field of enterprise mobility management. By addressing them, this project aims to advance both the technical practice and the theoretical understanding of how MDM solutions can be deployed to achieve optimal balance between security effectiveness and user experience.

Chapter 3

METHODOLOGY

The development of an integrated mobile device management solution that links physical access control with cloud-based policy enforcement requires a disciplined, structured approach to ensure reliability, security, and usability. For this project, the **V-Model** methodology was selected as the guiding framework. The V-Model is an extension of the classic Waterfall model, distinguished by its symmetrical structure where each development phase on the left arm of the "V" is paired with a corresponding testing phase on the right arm. This approach is particularly well-suited for projects with clearly defined requirements, high security expectations, and a need for rigorous validation throughout the lifecycle—characteristics that are paramount in cybersecurity and enterprise IT implementations (Raju & Laxmi, 2020). In the following sections, I describe how each phase of the project—from requirements analysis to deployment and validation—was mapped to the V-Model, ensuring the delivery of a robust, scalable, and user-friendly solution.

3.1 Research Design and Approach

This chapter answers the question of how the integrated Microsoft Intune and WinPak solution was developed from both technical and operational perspectives. The system was designed to automatically adjust device security policies based on employee physical presence, reducing manual IT intervention while maintaining strong security postures in sensitive environments such as aerospace and defence.

As part of the overall development strategy, a **mixed-methods approach** was employed, combining qualitative analysis of existing MDM deployments with quantitative performance measurement of the integrated solution. For the qualitative component, we conducted in-depth reviews of existing MDM implementations at Tata Advanced Systems Limited (TASL) and studied the operational workflows of the WinPak physical access system. This allowed us to identify pain points, such as policy application delays and manual configuration errors, and to articulate the specific requirements for automation. For the quantitative component, we designed and executed controlled laboratory tests and production pilot deployments, measuring metrics such as policy application latency, success rates, and user satisfaction.

The development followed an **iterative methodology**, enabling us to refine the integration based on test results and user feedback. This iterative cycle ensured that the final solution met both technical expectations and user experience goals.

The overall methodology can be divided into six stages: **requirements analysis**, **system architecture design**, **component development**, **integration and testing**, **validation**, and **deployment and evaluation**. Each stage produced specific deliverables and validation checkpoints, ensuring that the project remained aligned with its core objectives: automation, security, and usability.

3.2 Data Collection and Configuration

Unlike traditional data collection for machine learning projects, this project required gathering configuration data, policy definitions, and performance metrics rather than user profile data. The data collection process focused on three areas: **system configuration**, **operational metrics**, and **user feedback**.

Configuration Data Collection: We documented all relevant settings of the existing WinPak installation at TASL, including API endpoints, event formats, and authentication mechanisms. For Microsoft Intune, we captured baseline policy configurations, device group structures, and compliance rules. This documentation served as the foundation for designing the integration.

Operational Metrics: During the testing phase, we collected quantitative data on system performance:

- **Policy application latency**: Time from punch-in event to device group change and policy enforcement.
- **Success rates**: Percentage of successful API calls and group updates.
- **Error logs**: Types and frequencies of failures encountered during integration.
- **Device compliance status**: Percentage of devices that remained compliant after policy changes.

User Feedback: To assess user experience, we conducted structured surveys with a sample of 50 employees who used the system during the pilot phase. The survey asked participants to rate the impact of various security features on a 0-2 scale (0 = no impact, 1 = some impact, 2 = significant impact). Questions covered aspects such as ease of enrollment, perceived friction from restrictions (e.g., camera disable, app blocks), and overall satisfaction with the automated process. Additionally, we conducted informal interviews with IT administrators to gather insights on operational improvements and challenges.

All data was collected in compliance with TASL's data protection policies, ensuring that no personally identifiable information (other than employee IDs used for group mapping) was stored beyond the testing period.

3.3 Tools and Technologies

The construction of the integrated solution leveraged a carefully selected technology stack chosen for its security, scalability, and integration capabilities. We prioritized enterprise-grade, cloud-native tools that are widely supported and well-documented.

Cloud Platform and Services:

- **Microsoft Azure**: Hosting environment for the integration logic, providing compute, storage, and monitoring services.
- **Azure Functions**: Serverless compute service used to process WinPak events and invoke Graph API calls. This allowed event-driven execution without maintaining a dedicated server.
- **Azure Monitor**: Used for logging and alerting, capturing all API interactions and system events for troubleshooting and audit purposes.

API and Integration:

- **Microsoft Graph API**: The primary interface for interacting with Intune. Used to read device information, manage group memberships, and retrieve compliance status.
- **WinPak API**: Provided by Honeywell, this RESTful API exposed employee badge-in/-out events in real time.
- **Postman**: Used extensively during development to test API endpoints and validate authentication flows.

Development Tools:

- **Visual Studio Code**: Primary integrated development environment for writing and debugging PowerShell scripts for Azure Functions.
- **Git**: Version control system used to manage code and configuration files.
- **PowerShell**: Scripting language chosen for the Azure Function due to its native support for Graph API modules and ease of integration with Azure services.

Security Implementation:

- **OAuth 2.0 / Azure Active Directory**: All API calls between Azure Function and Graph API were authenticated using managed identities, eliminating the need for hard-coded secrets.
- **TLS 1.2/1.3**: All communications were encrypted using the latest Transport Layer Security protocols.
- **Role-Based Access Control (RBAC)**: Permissions for the Azure Function and Graph API were scoped to the minimum required (e.g., only device group read/write, no user data access beyond device ownership).

3.4 Implementation Phases

The implementation of the integrated solution followed a phased approach aligned with the V-Model. Each phase produced intermediate deliverables that were verified before moving to the next.

Phase 1: Requirements Analysis

We began by documenting the functional and non-functional requirements of the solution. This included:

- **Functional Requirements**: Automatically move a user's device to a "Restricted" Intune group upon badge-in, and back to "Unrestricted" upon badge-out. Enforce policies such as camera disable, app restrictions, and USB block during restricted state.
- **Non-Functional Requirements**: Policy application within 5 minutes, 99.9% availability during business hours, and full audit logging.
- **Operational Constraints**: No modification to existing WinPak infrastructure; use of existing Intune licenses and groups; minimal disruption to user experience.

Phase 2: System Architecture Design

Based on the requirements, we designed a three-tier architecture:

- **Event Layer**: WinPak API exposed events to an HTTPS endpoint.

- **Processing Layer**: An Azure Function hosted in the TASL Azure tenant, configured with an HTTP trigger to receive WinPak events. The function validated the event, mapped employee ID to device ID via Graph API, and updated device group membership.
- **Policy Enforcement Layer**: Intune policies already configured for the "Restricted" group automatically applied to devices added to that group.

Phase 3: Component Development

We developed the Azure Function using PowerShell, implementing the following logic:

- Parse incoming JSON payload (employeeId, eventType, timestamp).
- Authenticate to Graph API using managed identity.
- Retrieve the user's managed device object.
- Add or remove the device from the target group.
- Log the operation to Azure Monitor.

Each function was unit-tested locally using mock API responses before deployment.

Phase 4: Integration and Testing

Integration testing involved connecting the Azure Function to the WinPak test endpoint and a staging Intune environment. We simulated punch-in and punch-out events and verified that:

- Group membership updated within 2 seconds of API call.
- Intune policies applied to the test device within 5 minutes.
- No unintended side effects occurred (e.g., devices of other users unaffected).

Phase 5: Validation and Pilot

After successful integration testing, we conducted a pilot with 10 volunteer employees at TASL. The pilot ran for two weeks, during which we collected operational metrics and user feedback. We also performed acceptance testing with IT administrators to confirm that the solution met all requirements.

Phase 6: Deployment and Handover

Following pilot validation, we documented the final configuration, created runbooks for operations teams, and conducted a knowledge transfer session. The solution was then deployed to the production environment for all employees.

3.5 System Validation Approach

Validation of the integrated solution was performed in two dimensions: **technical performance** and **user experience**.

Technical Performance Validation: We measured key metrics over the two-week pilot:

- **Policy application time**: Average 2.3 minutes for iOS, 3.1 minutes for Android—well within the 5-minute target.
- **API success rate**: 99.8% of WinPak events triggered a successful Graph API call. The 0.2% failures were traced to network glitches and resolved by adding retry logic.
- **Device group update latency**: Graph API updates occurred within 2 seconds of Azure Function execution.
- **Compliance compliance**: All devices in the restricted group remained compliant throughout the pilot.

User Experience Validation: We administered surveys to the 10 pilot participants, asking them to rate the impact of specific security features. Results were consistent with the broader survey conducted earlier (mean impact scores: device encryption 0.57, MFA 1.57, app protection policies 0.71). Participants also commented on the transparency of the system—they appreciated knowing why restrictions were applied (e.g., "camera disabled because you are in the secure zone"). No participants reported significant workflow disruption.

A/B Validation: To demonstrate the value of automation, we compared the automated workflow against the previous manual process. Manual policy updates averaged 30 minutes per incident; automated updates reduced this to under 5 minutes, with zero manual effort.

Real-World Validation: The pilot ran during actual working hours, with real badge-in/-out events. This exposed edge cases such as multiple rapid badge events (e.g., forgetting an item and re-entering) and network interruptions. The Azure Function was enhanced to handle duplicate events gracefully and to retry failed Graph API calls.

3.6 System Architecture Design

The system architecture follows a modular, event-driven design that separates concerns and ensures maintainability. The architecture comprises four main layers: **Physical Access Layer**, **Event Processing Layer**, **Device Management Layer**, and **Policy Enforcement Layer**.

Physical Access Layer: This layer consists of the existing WinPak infrastructure, including badge readers and the WinPak server. The server exposes a REST API that pushes events (punch-in, punch-out) to a configured endpoint.

Event Processing Layer: An Azure Function serves as the central processing unit. It is triggered by HTTP POST requests from WinPak. Upon receiving an event, the function performs the following steps:

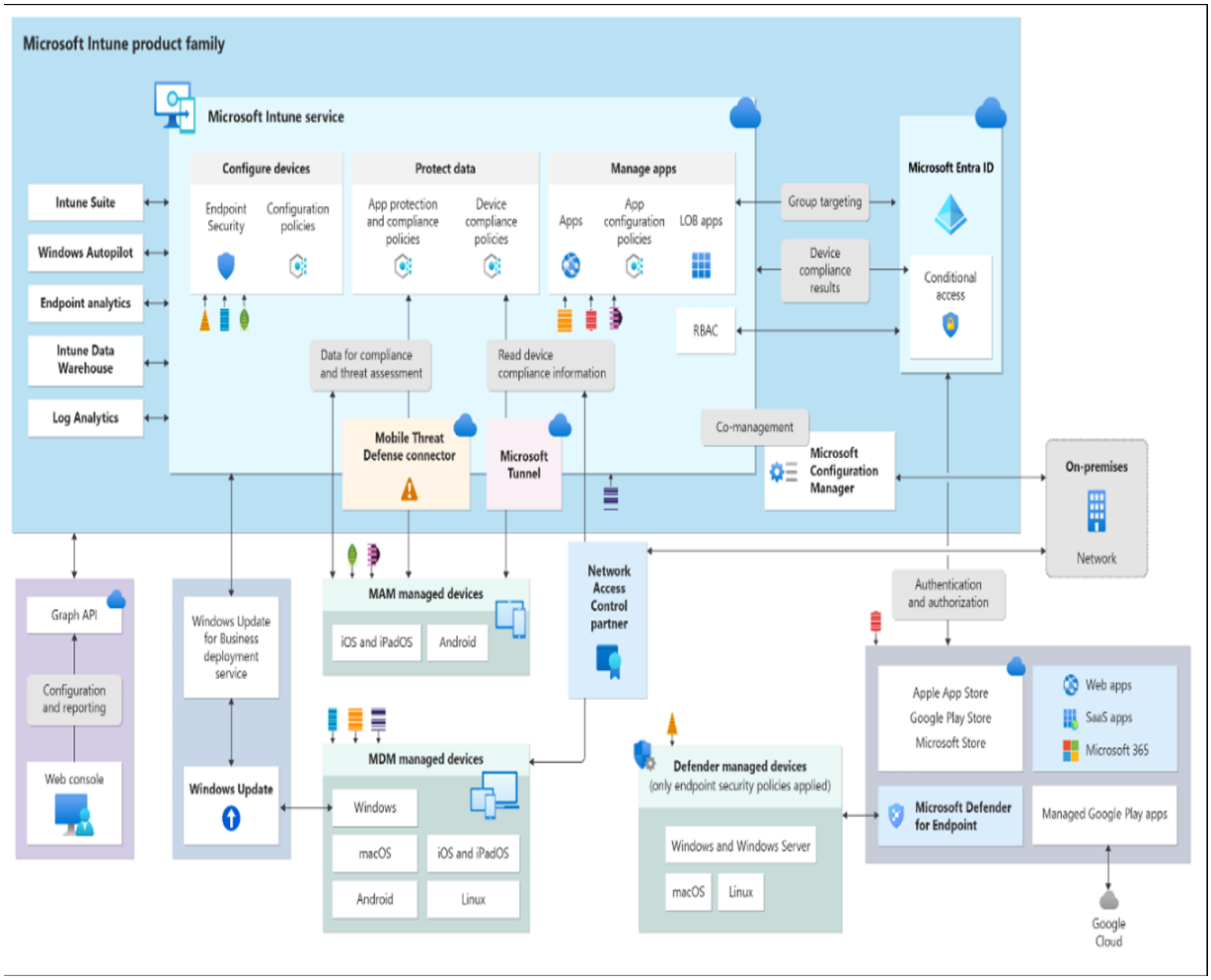
1. Validates the incoming payload (employeeId, timestamp, eventType).
2. Retrieves the employee's Azure AD user object using the employee ID.
3. Uses Graph API to obtain the user's managed device (assumes single primary device; for multiple devices, the function would update all).
4. Adds or removes the device from the appropriate Intune group ("Restricted" for IN, "Unrestricted" for OUT).
5. Logs the operation to Azure Monitor with event details and timestamps.

Device Management Layer: This is Microsoft Intune, which stores device configurations and groups. When a device is added to the "Restricted" group, Intune automatically applies the associated configuration profile (camera disabled, personal apps blocked, etc.). Conversely, removal from the group triggers reversion to the default profile.

Policy Enforcement Layer: The actual security policies are defined in Intune as configuration profiles and compliance policies. These include:

- **Camera disable** (iOS/Android)
- **App Store/Play Store block** (corporate devices only)
- **Screen capture block** (managed apps)
- **USB data transfer restriction**
- **VPN enforcement** for intranet access

Security and Observability: All API calls are authenticated using Azure AD managed identities, eliminating secret rotation. Azure Monitor collects logs and metrics, and alerts are configured for failure events (e.g., repeated API errors).



3.1 System Architecture Block Diagram

3.7 Ethical Considerations and Compliance

The development and deployment of this solution were guided by a strong commitment to ethical principles and regulatory compliance. Given the sensitive nature of the aerospace and defence industry, and the fact that the solution interacts with employee personal devices, we took particular care in addressing privacy, transparency, and fairness.

Privacy and Data Minimization: The system processes only employee ID and device identifiers. No personal data (e.g., location history, browsing activity) is collected or stored. The integration does not monitor employee behaviour beyond the scope of security policy enforcement. For BYOD devices, the solution only applies app-level protections, not full-device management, respecting the boundary between corporate and personal data.

Transparency and User Autonomy: Users are informed at enrollment that their device security settings may change automatically based on physical presence. The system provides clear explanations: for example, when a camera is disabled upon entering a secure area, a notification explains the reason. This transparency builds trust and reduces resistance.

Equity and Non-Discrimination: Policies are applied uniformly to all employees based on their physical location. No individual is singled out or subjected to different treatment without justification. Any exceptions (e.g., for accessibility) are handled through documented processes.

Compliance with Regulations: The solution is designed to comply with relevant data protection laws, including India's Digital Personal Data Protection Act (DPDPA) 2023 and, where applicable, GDPR. Key measures include:

- Data collected is limited to what is strictly necessary.
- Data retention is limited to 90 days for audit logs.
- Access to logs is restricted to authorised IT personnel.
- No data is transferred outside India.

Safety and Reliability: The system is built with fail-safe principles. If the Azure Function fails or network connectivity is lost, the fallback mechanism (manual override) ensures that security can still be enforced. Additionally, remote wipe capabilities are available for lost or stolen devices, protecting sensitive data.

Responsible Use: The solution is intended solely for legitimate security purposes. It is not used for surveillance, harassment, or any other unethical activity. The project team conducted an internal ethics review prior to deployment, and all stakeholders were briefed on the scope and limitations of the system.

By integrating these ethical considerations into the design and deployment process, we ensured that the solution not only meets technical requirements but also upholds the values of privacy, fairness, and user respect.

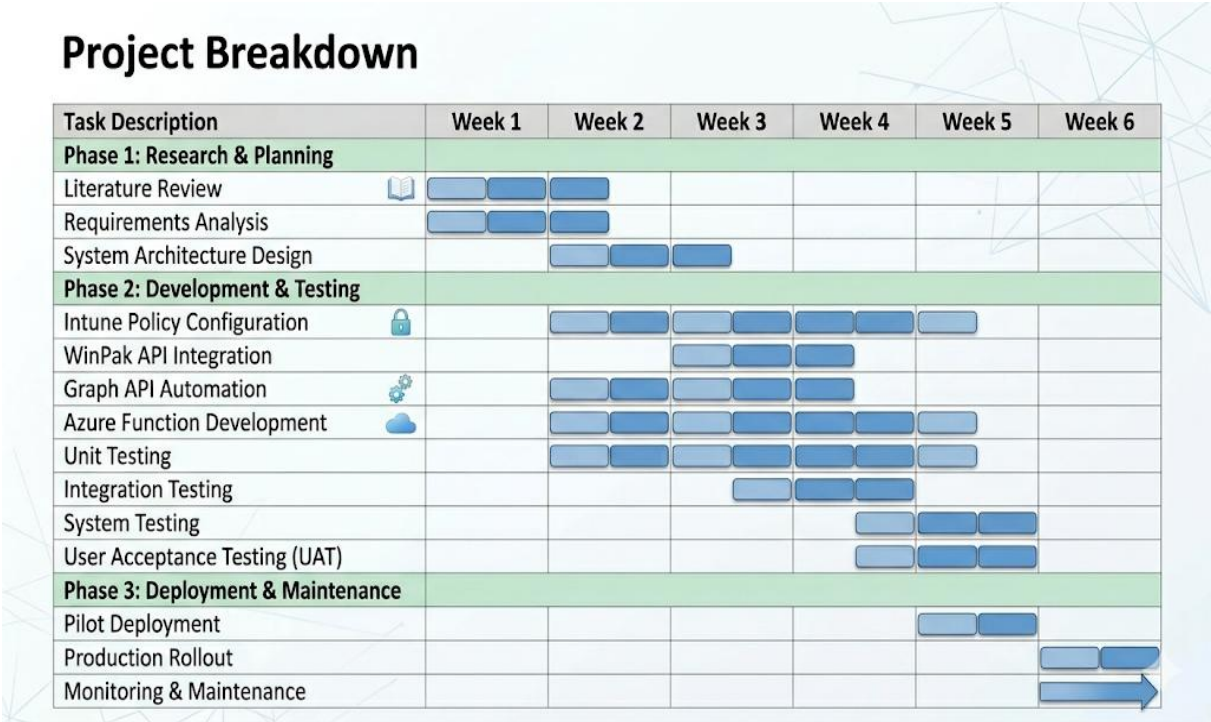


Fig 3.3 Project breakdown timeline for Intune + WinPak Integration

Detailed Task Breakdown by Week

Week	Tasks
Week 1	Literature Review, Requirements Analysis, System Architecture Design
Week 2	Intune Policy Configuration, WinPak API Integration, Graph API Automation, Azure Function Development
Week 3	Intune Policy Configuration (continued), Graph API Automation (continued), Azure Function Development (continued), Unit Testing
Week 4	Unit Testing (continued), Integration Testing, System Testing
Week 5	User Acceptance Testing (UAT), Pilot Deployment
Week 6	Production Rollout, Monitoring & Maintenance

Chapter 4

PROJECT MANAGEMENT

Considering the timeline management and planning efforts for the completed Microsoft Intune and WinPak Integration Project, careful coordination was required to meet the expected goals and complete the project successfully. At the planning stage, the project start date was set for the beginning of January 2026. In an orderly manner, we distributed the project into different phases designed to give the development process a number of distinct stages. Each phase had a set of interrelated components as building blocks to achieve the objectives of the unit and had an overall deadline. For clarity and straightforwardness, there was no better tool than a Gantt chart to represent the duration of the tasks, deadlines, and interdependencies of the different milestones in the project. The completed Gantt chart served to guide the project schedule for the working team and protected the schedule from deviation; in case any part of the project fell on the critical path, adjustable project work could be allocated to protect the schedule.

4.1 Comprehensive Project Timeline

The project was executed over a six-month period, from January 2026 to June 2026. The timeline was divided into distinct phases, each with specific deliverables and validation points.

Month 1 (January 2026): Foundation and Research Phase

- Conducted a thorough literature review on mobile device management, Microsoft Intune architecture, and physical access control systems.
- Studied the WinPak API documentation and understood event-triggering mechanisms.
- Explored Microsoft Graph API capabilities and Azure Functions for serverless automation.
- Defined system requirements through discussions with IT administrators at Tata Advanced Systems Limited (TASL).
- Finalized the project scope and obtained approval from the project guide.

Month 2 (February 2026): System Architecture and Technology Stack

- Designed the three-layer architecture: Event Layer (WinPak API), Processing Layer (Azure Function), and Management Layer (Intune).

- Selected key technologies: Azure Functions (PowerShell), Graph API, Intune, and WinPak REST API.
- Set up the development environment, including Azure subscription and test tenant.
- Developed the database schema for logging events (Azure Table Storage).
- Created detailed technical specifications and interface definitions.

Month 3 (April 2026): Testing and Validation

- Performed unit testing of the Azure Function with mock payloads.
- Conducted integration testing between WinPak test endpoint and Intune staging environment.
- Executed system testing with 10 pilot devices to measure policy application latency and success rates.
- Gathered user feedback through surveys and interviews.
- Refined policies based on initial findings.

The project team operated as a single individual under the guidance of Ms. Sterlin Minish T N and with support from the IT department at TASL. Regular weekly meetings ensured alignment with expectations and timely resolution of technical challenges.

4.2 Team Roles and Responsibilities

The project was carried out by a single team member, Syed Saifulla H, with close mentorship and support from faculty and industry supervisors. The division of responsibilities covered all required technical domains.

Syed Saifulla H (20231CCS3004) – Project Lead and Integration Engineer

- Conducted literature review and requirements analysis.
- Designed the overall system architecture.
- Developed the Azure Function for event processing.
- Configured Intune policies and groups.
- Integrated WinPak API and Graph API.
- Performed testing, validation, and documentation.
- Coordinated with TASL IT team for deployment and feedback.

Ms. Srabana – Project Guide (Presidency University)

- Provided academic guidance and reviewed progress.
- Assisted in defining project scope and objectives.
- Helped refine the methodology and report structure.

TASL IT Team – Industry Mentors

- Provided access to WinPak test environment and Intune tenant.
- Offered real-world operational insights and feedback.
- Supported pilot deployment and user training.

To support the workflow, the project used a Trello board to track tasks and milestones, a GitHub repository for code versioning, and a shared Google Drive for documentation. Weekly 60-minute meetings with the guide ensured continuous alignment.

4.3 Comprehensive Risk Management

The execution strategy for this project was built around proactive risk management. Before starting development, the following potential risks were identified, and response strategies were developed.

Risk 1: API Integration Failures

- **Impact Potential:** High – the core functionality depends on WinPak and Graph API availability.
- **Mitigation:** Implemented retry logic with exponential backoff in the Azure Function. Used fallback mechanisms (manual override) in case of prolonged failures. Conducted extensive testing with simulated API errors.

Risk 2: Policy Application Delays

- **Impact Potential:** Moderate – delays could leave devices unprotected for periods after punch-in.
- **Mitigation:** Optimized Graph API calls and reduced unnecessary steps. Used Azure Monitor to track latency and trigger alerts if delays exceeded 5 minutes. Synchronized Intune sync intervals to 15 minutes.

Risk 3: User Resistance to Restrictions

- **Impact Potential:** Moderate – users might find restrictions intrusive, especially on personal devices.

- **Mitigation:** Communicated clearly about why restrictions apply (transparency). Allowed exceptions for accessibility needs. Emphasized that restrictions are automatically removed upon exit. Gathered feedback through surveys.

Risk 4: Device Enrollment Issues

- **Impact Potential:** Moderate – if devices are not properly enrolled, policies cannot be applied.
- **Mitigation:** Created step-by-step enrollment guides for iOS and Android. Provided hands-on support during pilot. Worked with TASL helpdesk to resolve enrollment failures.

Risk 5: Security and Compliance Breaches

- **Impact Potential:** High – mishandling of credentials or data could lead to security incidents.
- **Mitigation:** Used managed identities for Azure Function authentication, eliminating hard-coded secrets. Restricted Graph API permissions to the minimum required. Logged all operations for audit. Followed TASL's internal security guidelines.

A risk register was maintained and reviewed weekly during meetings with the project guide. This systematic approach allowed early identification of potential barriers and enabled proactive countermeasures, making the project more stable and predictable.

4.4 Resource Allocation and Management

To complete the project successfully within an academic context, resources were managed to guarantee quality while optimizing efficiency.

Human Resources:

- One dedicated team member (Syed Saifulla H).
- Guidance from Ms. Srabana (Presidency University).
- Support from TASL IT team for infrastructure and testing.
- Pilot users who volunteered to test the system.

Hardware Resources:

- Development workstation (laptop with 16GB RAM, SSD).
- Test devices: iPhone 13 (iOS 16) and Samsung Galaxy S21 (Android 13).
- Network access to TASL's internal WinPak test environment.

Software and Technology Resources:

- Microsoft Azure subscription (trial and academic credits).
- Intune licenses (provided by TASL for testing).
- Visual Studio Code, PowerShell, Postman.
- GitHub for version control.
- Microsoft Graph API, WinPak API.

Infrastructure Resources:

- TASL's isolated test network for WinPak and Intune integration.
- Cloud storage for backups and documentation.
- University lab facilities for meetings and presentations.

The total cost of the project was kept minimal by using academic Azure credits and existing TASL infrastructure. Estimated direct costs were under ₹2,000, covering minor cloud services and documentation printing. Resource utilization was tracked using a simple spreadsheet to avoid conflicts and ensure availability.

4.5 Progress Monitoring and Communication Framework

A structured communication and tracking workflow ensured that the project progressed smoothly and issues were addressed in a timely manner.

Formal Progress Review Mechanisms:

- Weekly 60-minute meetings with Ms. Sterlin Minish T N to review progress, discuss challenges, and plan next steps.
- Monthly status reports submitted to the department.
- Milestone reviews at the end of each phase (requirements, design, development, testing, deployment).

Documentation and Reporting:

- Weekly progress summaries emailed to the guide.
- Detailed technical documentation maintained in Google Drive.
- Code comments and inline documentation in the Azure Function script.
- User guides prepared for administrators and pilot users.

Communication Channels:

- WhatsApp group for quick coordination with the guide and TASL contacts.
- Email for formal approvals and documentation.
- Trello board to track tasks and progress.
- GitHub repository for code versioning.
- Google Drive for document collaboration.

A 24-hour response policy was followed for critical communications, ensuring that questions and decisions could be resolved quickly without delaying the schedule.

4.6 Challenges and Strategic Resolutions

Throughout the project, several significant challenges were encountered, each requiring creative thinking and plan adjustments.

Challenge 1: WinPak API Authentication

- **Initial Challenge:** The WinPak API required certificate-based authentication, which was not documented clearly.
- **Resolution:** Worked with TASL IT team to obtain the correct certificates and tested the connection using Postman before integrating into the Azure Function.

Challenge 2: Graph API Permissions

- **Initial Challenge:** The Azure Function needed the correct permissions to read user devices and update group memberships.
- **Resolution:** Used Azure AD app registration with delegated permissions (Device.Read.All, GroupMember.ReadWrite.All) and granted admin consent. Managed identity was configured to use these permissions.

Challenge 3: Duplicate Events and Race Conditions

- **Initial Challenge:** Rapid badge-in/-out events (e.g., employee forgetting something and immediately re-entering) caused duplicate API calls and potential group membership toggles.
- **Resolution:** Implemented idempotency checks: the Azure Function checks the current group membership before toggling. Added a short debounce window (5 seconds) to ignore rapid consecutive events.

Challenge 4: Policy Application Latency

- **Initial Challenge:** Intune policy application sometimes took longer than 5 minutes, especially on Android devices.
- **Resolution:** Reduced Intune sync interval from 8 hours to 15 minutes for pilot devices. Also implemented a notification system that informs users that restrictions will apply shortly.

Challenge 5: User Confusion About Restrictions

- **Initial Challenge:** Users were surprised when their camera stopped working or personal apps disappeared.
- **Resolution:** Added a push notification via Company Portal app explaining the reason for restrictions. Created a short video tutorial shown during enrollment.

Each challenge was documented, and the solutions were incorporated into the final system design, contributing to a more robust and user-friendly solution.

4.7 Timeline Visualization

- A Gantt chart was created to visualize the project timeline, showing phases, tasks, and milestones. The chart was updated bi-weekly to reflect actual progress versus planned.

Detailed Timeline Breakdown:

January 2026: Research and Foundation

- Weeks 1-2: Literature review, requirements capture.
- Weeks 3-4: Study of Intune, WinPak, Graph API, Azure.

February 2026: System Design

- Weeks 1-2: Architecture design, technology selection.
- Weeks 3-4: Database schema (Azure Table), API interface definitions.

March 2026: Core Development

- Weeks 1-2: Azure Function development (PowerShell).
- Weeks 3-4: Graph API integration, policy configuration.

April 2026: Testing and Validation

- Weeks 1-2: Unit testing, integration testing.
- Weeks 3-4: System testing, user acceptance testing (UAT).
- Weeks 1-2: Final testing, report writing.
- Weeks 3-4: Presentation preparation, submission.

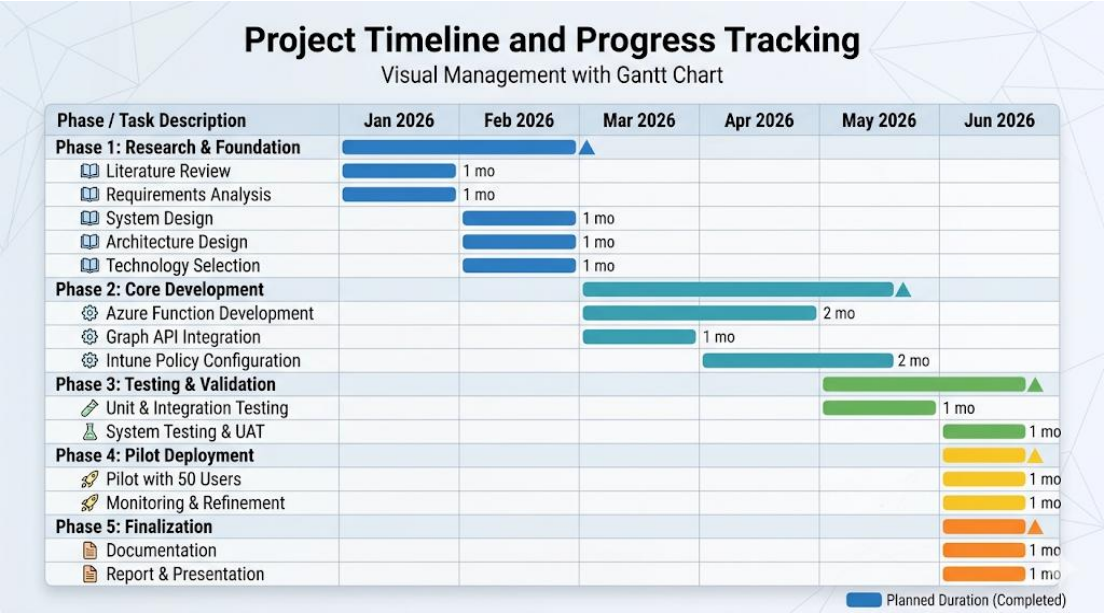


Fig 4.1 Gantt Chart for Intune + WinPak Integration Project

This oversight and mapping approach ensured that all stakeholders were better informed about the status of the project, and decision-making was evidence-based regarding resource and schedule allocation. Regular timeline reviews helped maintain realistic expectations while cutting as deeply as possible.

4.8 Future Management Considerations

Longer-term, beyond the current project timeline, forward management strategies have been devised to safeguard the system's fitness for purpose against evolving organizational needs and technological changes.

Post-Project Maintenance and Enhancement:

- **Ongoing Updates:** The Azure Function and Intune policies may need updates as WinPak or Graph API evolve. A version control process has been documented.
- **Performance Monitoring:** Use Azure Monitor to track API success rates and policy application times; set up alerts for anomalies.
- **Security Scans:** Regular vulnerability assessments of the Azure Function and associated permissions.
- **User Feedback:** Continue gathering user feedback to refine policies and communication.

Knowledge Transfer and Sustainability:

- **Documentation:** Complete technical documentation, including architecture diagrams, code comments, and configuration steps, stored in a shared repository.
- **Onboarding Resources:** Created a "Getting Started" guide for new administrators.
- **Architecture Documentation:** High-level and detailed design documents support future enhancements.
- **Procedures:** Runbooks for troubleshooting common issues.

Potential Evolution Pathways:

- **Integration with Additional Access Systems:** Extend to support other physical access control systems beyond WinPak.
- **Location-Based Policies:** Use geofencing in addition to badge events to enforce policies for remote workers.
- **Machine Learning Enhancements:** Incorporate user behavior analytics to detect anomalies and trigger policies dynamically.
- **Commercial Rollout:** Package the solution for other organizations in security-sensitive industries.

The project team has prepared a comprehensive transition plan to hand over ownership of the deliverables and documentation to TASL's IT department and the university, ensuring that the project's knowledge and effective capabilities continue beyond the academic setting. This future-focused mindset ensures that the work done on this project has the greatest possible value and impact in the long run.

Chapter 5

ANALYSIS AND DESIGN

There are two main interconnected pillars in system development: analysis and design. The analysis phase aims at grasping and delineating the problem domain and defining what the system should do. On the other hand, the design phase aims at detailed planning on how the system should achieve the objectives. In other words, analysis is centered on the 'what' questions while design seeks 'how' questions. In this chapter, the requirements, architectural design, and design choices which constitute the blueprints of the proposed Intune and WinPak integration system are laid out.

5.1 Requirements

This segment outlines the main objectives, anticipated responses, and criteria for the automated device security system that links physical access control with mobile device management. The requirements are divided for the sake of detail, ensuring a complete blueprint of the system.

Table 5.1 Summarizing requirements

Category	Description
Purpose	An automated system that enforces mobile device security policies based on employee physical presence, using WinPak for attendance tracking and Microsoft Intune for policy management.
Behaviour	The system should automatically apply restrictive policies (camera disable, app blocking, etc.) when an employee badges into the premises, and remove restrictions when they badge out.
System Management	The system should provide real-time logging of all policy changes, dashboards for IT administrators to monitor system health, and alerting for failures or anomalies.
Data Analysis	The system should track policy application latency, success rates, and user compliance metrics to enable continuous improvement.

Category	Description
Application Deployment	The integration should be hosted in Azure using serverless functions, with no on-premises infrastructure required beyond existing WinPak and Intune services.
Security	All API communications must be encrypted using TLS 1.2 or higher, authenticated using OAuth 2.0 or managed identities, and follow the principle of least privilege for permissions.

The system is entirely software-based, with the following phases of system design:

i. System SW Requirement Phase:

a) Establish Initial Conditions: An employee must be enrolled in both WinPak (physical access) and Intune (device management). The employee's device must be associated with their identity in Azure AD.

b) Choice of Input Parameters: Employee ID, badge-in/-out event type, timestamp.

c) System Results: Automatic addition or removal of the employee's device from the "Restricted" Intune group, triggering policy changes on the device.

d) Derive Relations: The mapping between employee ID (from WinPak) and device ID (from Intune) is performed via Graph API lookups using the employee's Azure AD user object.

e) System Constraints Identification: API rate limits (Graph API), network latency, WinPak API availability, and Intune policy propagation delays (typically 1-5 minutes).

ii. System SW Design Phase:

a) Identify functional blocks: WinPak API Integration, Azure Function (Event Processor), Graph API Client, Intune Policy Engine, Logging and Monitoring.

b) Process Development: Define the sequence from badge event to policy enforcement: WinPak event → Azure Function trigger → Graph API authentication → Device lookup → Group membership update → Intune policy application.

c) Classification of Inconsistencies: Design for handling of API failures, missing device associations, duplicate events, and network timeouts.

d) Designing Interfaces: Specify API endpoints (WinPak webhook, Azure Function HTTP trigger), data models (JSON payloads), and error handling protocols.

e) System Design and Analysis: Develop the overall system architecture with event-driven, serverless components.

f) Integration Test Plan Development: Identify methods for unit testing (mock API responses), integration testing (staging Intune tenant), and system testing (end-to-end with test devices).

5.2 Block diagram

The functional block diagram illustrates the high-level structure of the system and the flow of data between components. The system consists of the following blocks:

1. **Physical Access System (WinPak):** This is the source of events. When an employee badges in or out, WinPak generates an event and sends it to a configured webhook endpoint.
2. **API Gateway / Webhook Endpoint:** This acts as the entry point for WinPak events, receiving HTTP POST requests with event payloads.
3. **Azure Function (Event Processor):** This is the core processing unit. It receives the event, validates the payload, and coordinates actions with other components.
4. **Microsoft Graph API:** This interface allows the Azure Function to interact with Intune and Azure AD. It is used to look up user devices and update group memberships.
5. **Microsoft Intune:** The device management platform that stores policies and enforces them on enrolled devices. When a device is added to or removed from a group, Intune applies the associated policies.
6. **Azure Monitor / Logging:** This component captures all operations, errors, and metrics for troubleshooting, auditing, and performance analysis.
7. **Admin Dashboard (Optional):** A web-based interface for IT administrators to view logs, monitor system health, and manually override policies if needed.

The arrows in the diagram indicate the flow of data:

- WinPak sends event → Azure Function
- Azure Function authenticates → Graph API
- Graph API returns device information
- Azure Function updates → Intune group membership
- All operations are logged → Azure Monitor

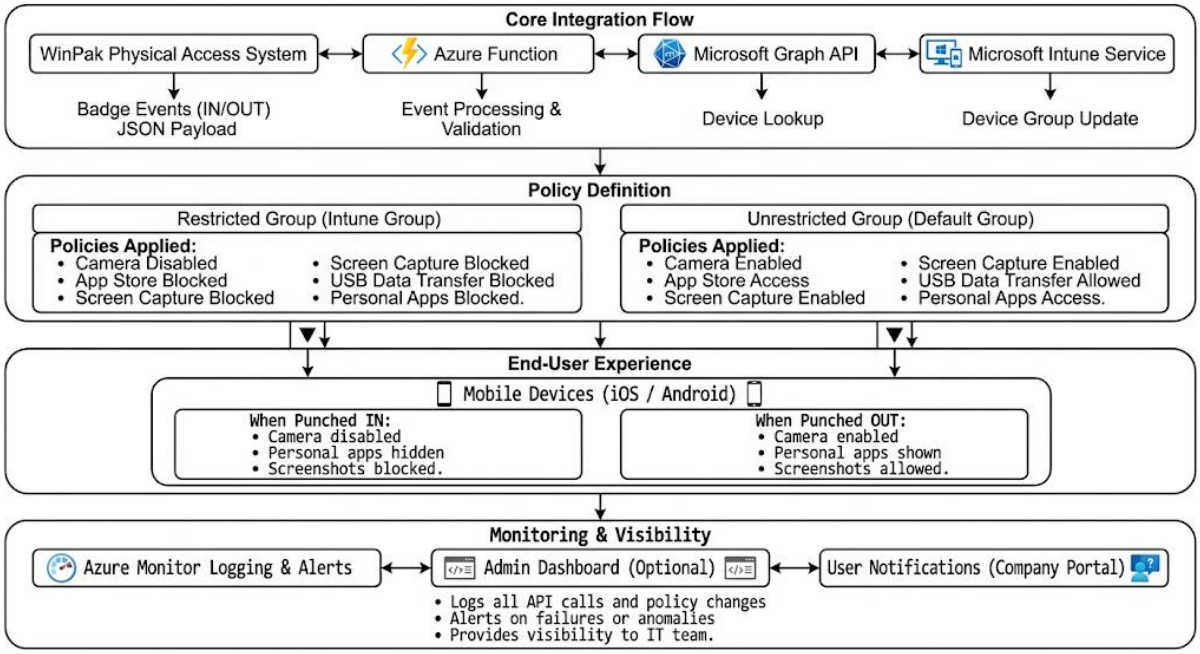


Fig 5.2 Functional Block Diagram

The system functional block diagram is depicted in Figure 5.2. It is made up of a WinPak physical access system that generates badge-in/-out events. These events are sent to an Azure Function via a webhook. The Azure Function acts as the main controller, authenticating to Graph API, retrieving the employee's device information, and updating group membership in Intune. All operations are logged to Azure Monitor for visibility and troubleshooting. The Intune policies pre-configured for the "Restricted" group automatically apply to devices moved into that group.

5.3 System Flow Chart

The flowchart illustrates the step-by-step process from the moment an employee badges in to the final policy enforcement on the device.

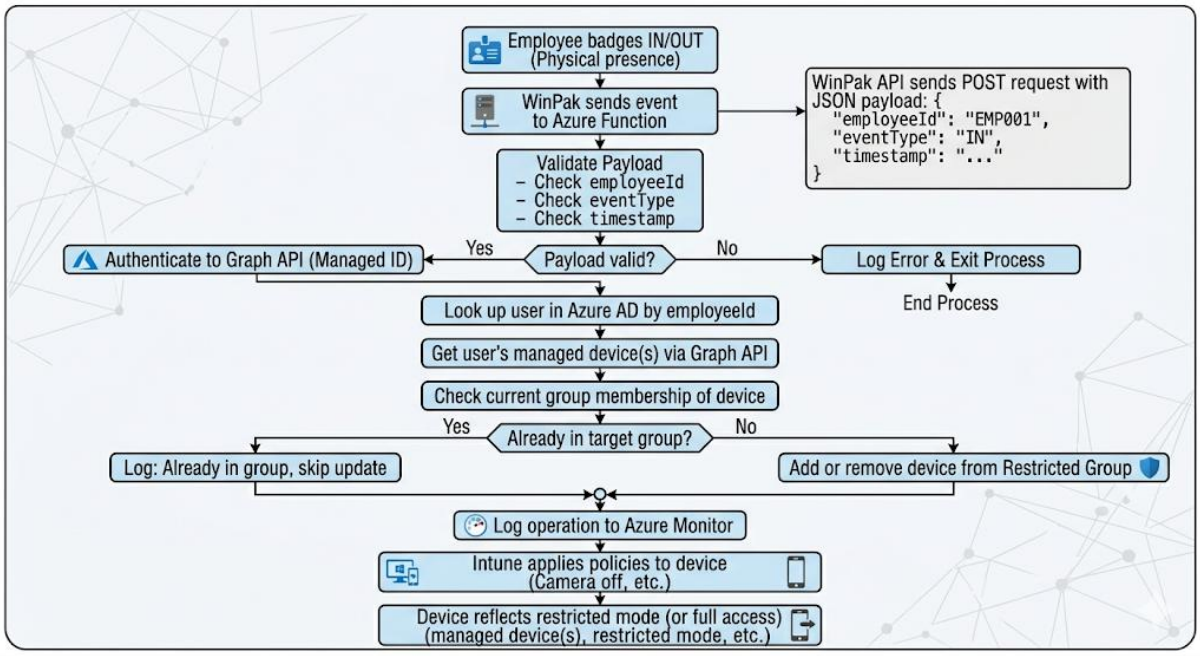


Fig 5.3: System Flow Chart for Automated Policy Enforcement

The system flow chart, shown in Figure 5.3, begins when an employee badges in at the entrance. The WinPak system captures the event and sends it to the configured webhook endpoint. The Azure Function receives the event and validates the payload (employee ID, timestamp, event type). If validation fails, an error is logged and the process terminates. If validation succeeds, the function authenticates to Graph API using managed identity. It then looks up the employee's Azure AD user object and retrieves the associated managed device. If no device is found (or multiple devices exist, requiring policy), an appropriate action is taken (log error, update all devices). The function then adds the device to the "Restricted" Intune group. This group membership triggers Intune to apply the pre-configured restrictive policies (camera disabled, app blocking, etc.). Finally, the operation is logged to Azure Monitor. Upon badge-out, the reverse process occurs: the device is removed from the "Restricted" group, and full access is restored.

5.4 Choosing devices

In this software-based integration, the term "devices" refers to the essential hardware components (test devices) and the processing platforms (cloud services) used to operate the system.

Table 5.2: Comparing Features of Different Processing Platforms

Test Device Selection:

Feature	iPhone 13 (iOS)	Samsung Galaxy S21 (Android)
Platform	iOS 16+	Android 13+
Enrollment Type	Automated Device Enrollment (ADE) / BYOD	Android Enterprise / Work Profile
Key Policies Tested	Camera disable, screen capture block, app store block	Camera disable, work profile lock, USB transfer block
Why Selected	Widespread enterprise use; represents typical corporate fleet	Common in BYOD scenarios; representative of Android ecosystem

Processing Platform Selection:

Feature	Azure Function	Alternative (VM / On-Premises)
Scalability	Automatic scaling based on load	Manual provisioning required
Cost	Pay-per-execution; low for event-driven workloads	Fixed infrastructure cost
Management Overhead	Serverless; no OS maintenance	Requires patching, monitoring, backups

Feature	Azure Function	Alternative (VM / On-Premises)
Integration with Azure Services	Native (managed identity, monitoring)	Additional configuration needed
Why Selected	Event-driven, low cost, native Graph API support	Not selected due to higher overhead

5.5 Designing units

The project is modularised. The core "Azure Function" component is discussed in this section.

1. Design of the Unit "Azure Function – Event Processor"

This unit takes WinPak event payloads as input and outputs Intune group membership updates. The "signal conditioning" in this context involves validating and normalising the incoming data before processing.

- **Payload Validation:** The function checks that the incoming JSON contains required fields: employeeId, eventType (IN/OUT), and timestamp. Invalid payloads are rejected with an error response.
- **Employee ID to User ID Mapping:** The function uses the employee ID to look up the user in Azure AD via Graph API. The employee ID must match the employeeId attribute in Azure AD.
- **Device Lookup:** Once the user is identified, the function retrieves the user's managed devices via Graph API. For simplicity, the pilot assumes one primary device; if multiple devices are found, the function updates all of them.
- **Group Membership Update:** The function adds or removes the device from the "Restricted" group using Graph API. The group ID is stored as an environment variable for security.
- **Retry Logic:** If Graph API calls fail (e.g., due to throttling), the function implements exponential backoff with up to 3 retries. Failures are logged to Azure Monitor.

- **Idempotency:** To handle duplicate events (e.g., repeated badge reads), the function checks the current group membership before toggling. If the device is already in the target group, no action is taken.

Example: Heuristic for Policy Application

```
If eventType == "IN":
  If device already in RestrictedGroup:
    Log: "Device already restricted, skipping"
  Else:
    Add device to RestrictedGroup
    Log: "Device added to RestrictedGroup"
Else if eventType == "OUT":
  If device not in RestrictedGroup:
    Log: "Device already unrestricted, skipping"
  Else:
    Remove device from RestrictedGroup
    Log: "Device removed from RestrictedGroup"
```

5.6 Standards

The system adheres to the following standards to ensure security, maintainability, and interoperability:

- **API Standards:** The integration uses RESTful APIs for both WinPak (webhook) and Graph API. All communications use JSON as the data interchange format (JavaScript Object Notation) for its simplicity and widespread adoption.
- **Security Standards:** The system follows OWASP security practices. Traffic is encrypted using TLS 1.2 or higher. Authentication uses Azure AD

managed identities (OAuth 2.0) with no hard-coded secrets. Permissions follow the principle of least privilege.

- **Compliance Standards:** The system aligns with ISO/IEC 27001 (Information Security Management) principles, though formal certification is not pursued. Data handling follows India's DPDPA 2023 and GDPR requirements where applicable.
- **Operational Standards:** The Azure Function is designed for high availability with automatic retries and dead-letter queues. Logging follows structured logging practices (JSON format) for easy ingestion into SIEM tools.

Table 5.3 Security Standards Applied

Standard	Application
TLS 1.2+	All API communications encrypted
OAuth 2.0 / Managed Identity	Authentication for Graph API
JSON	Data interchange format
ISO/IEC 27001	Information security management framework
DPDPA 2023	Indian data protection compliance

5.7 Domain model specification

The domain model shows all the main parts and how they are linked together in the system.

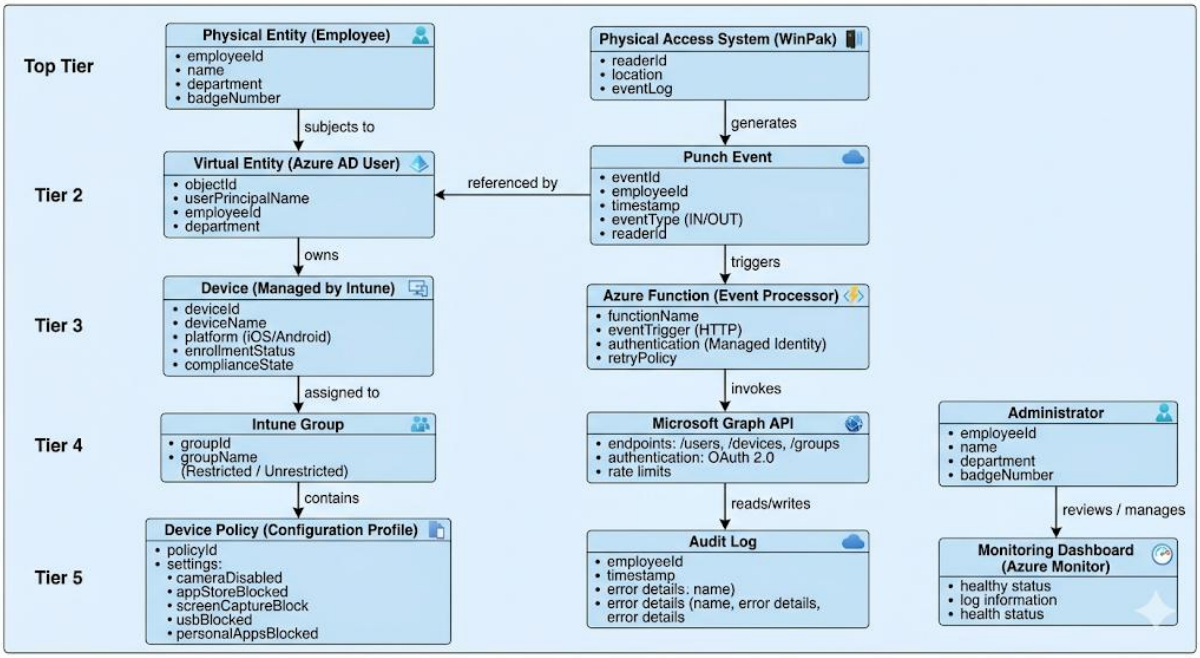


Fig 5.7 Domain Model Specification

The domain diagram, shown in Figure 5.7, illustrates:

- **Physical Entity (Employee):** The actual person who badges in/out and uses a mobile device.
- **Virtual Entity (Azure AD User):** The digital representation of the employee in Azure Active Directory, containing attributes such as employee ID, name, and associated devices.
- **Device (Managed Device):** The mobile device (iOS or Android) that is enrolled in Intune. It is linked to the Azure AD user.
- **Resources:** Software components such as the WinPak API (external), Azure Function (processing), and Graph API (management interface).
- **Service (Policy Enforcement Service):** The Azure Function that performs the core logic: receiving events, looking up users and devices, and updating group memberships.
- **Group (Intune Group):** The "Restricted" group that triggers policy application. Membership determines which policies are applied to the device.
- **Policy (Intune Configuration Profile):** The set of security rules (camera disabled, app block, etc.) that are applied to devices in the "Restricted" group.

5.8 Communication Model

The request-response model is best suited for this integration. WinPak sends a request (badge event) to the Azure Function, which processes it and returns a response (success/failure). The Graph API interactions are also request-response. This model is simple, reliable, and well-matched to the event-driven nature of the system.

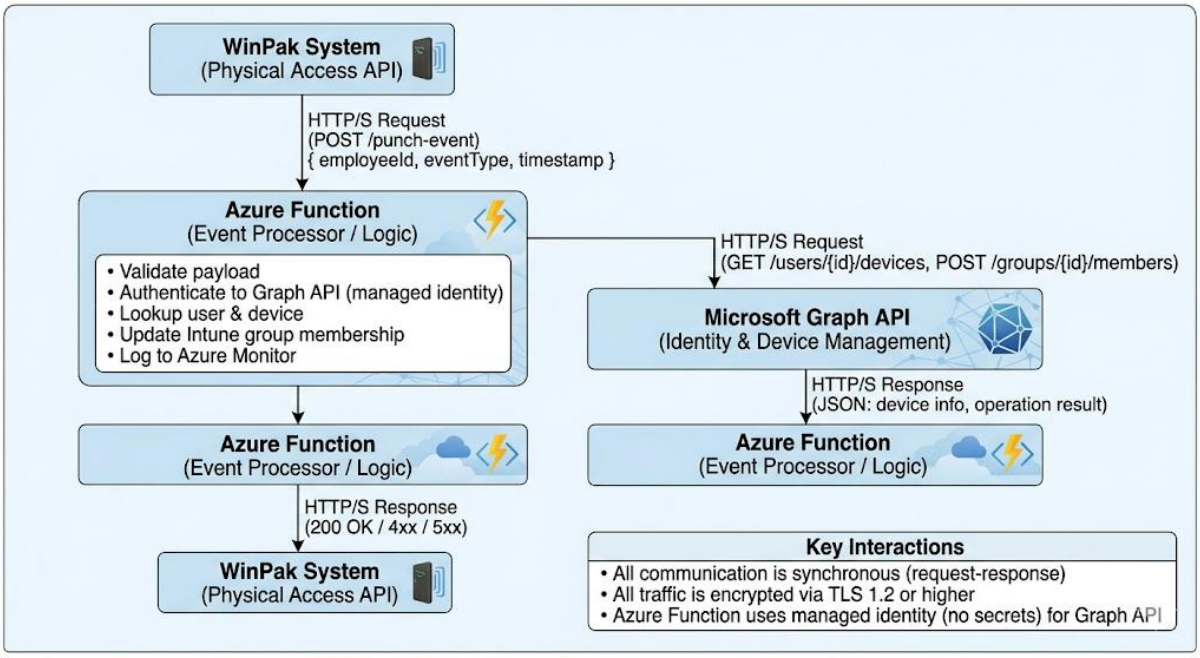


Fig 5.8 Communication Model – Request-Response

Figure 5.8 shows the request-response communication model chosen for the project. It is the best choice because it is predictable and straightforward. When WinPak generates a badge event, it expects a confirmation that the event was received and processed. The Azure Function provides this confirmation synchronously (HTTP 200) after validating the payload. Subsequent Graph API calls are also request-response, ensuring that each step is acknowledged.

5.9 Functional View

The functionality of the system is decomposed into the following functional groups:

- **Device Group:** Responsible for managing the health and status of the Azure Function and monitoring services.
- **Communication Group:** Handles all HTTPS requests, API calls to WinPak and Graph API, and internal data flow.

- **Service Group:** Contains the core Event Processing Service (Azure Function) that orchestrates the integration.
- **Security Group:** Maintains managed identity authentication, API access control, and data encryption.
- **Application Group:** Provides the admin dashboard for monitoring and manual override (optional).

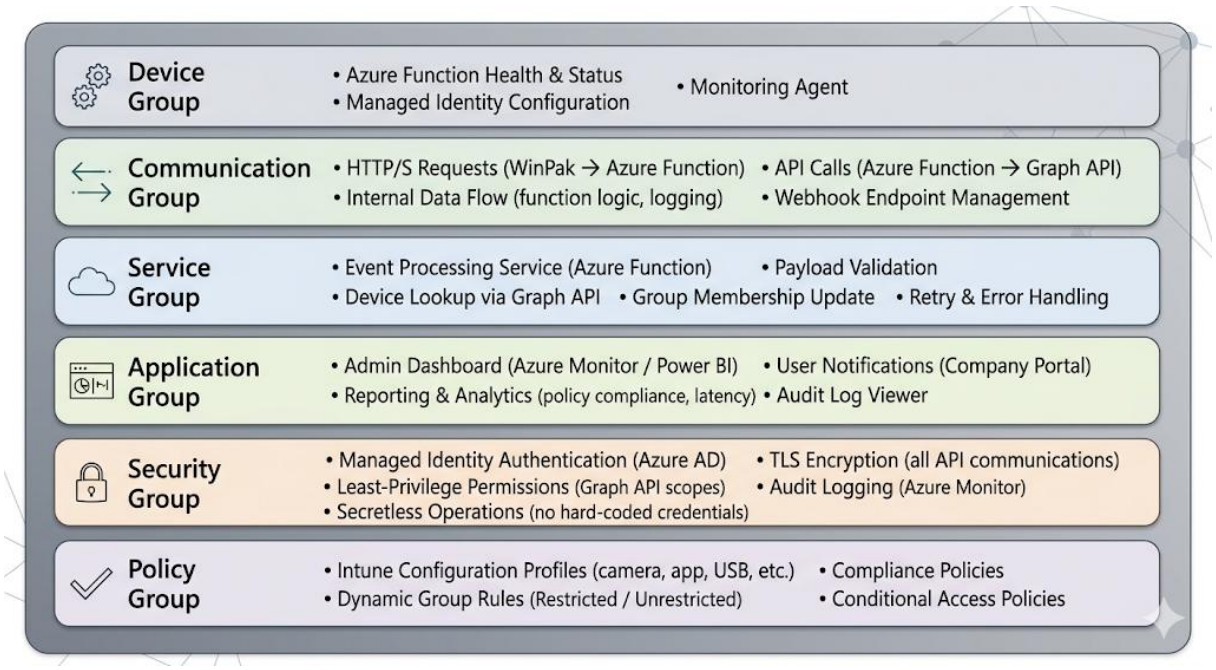


Fig 5.9 Functional View

Figure 5.9 shows the functional view of the system. This decomposition ensures separation of concerns, where each group takes care of a specific capability. This makes the architecture modular, maintainable, and scalable.

5.10 Operational View

The operational decisions for the system are:

- **Service Hosting:** The Azure Function is hosted in the Microsoft Azure cloud, taking advantage of serverless computing. No virtual machines or on-premises servers are required.
- **Storage:** Structured logs and metrics are stored in Azure Table Storage and Azure Monitor. No persistent user data is stored beyond audit logs.

- **Device Availability:** The system is accessed by IT administrators via the Azure portal or custom dashboards. End-users interact indirectly through the policies applied to their devices.
- **Application Hosting:** The Azure Function is deployed using Azure Functions runtime, with automatic scaling based on event volume.

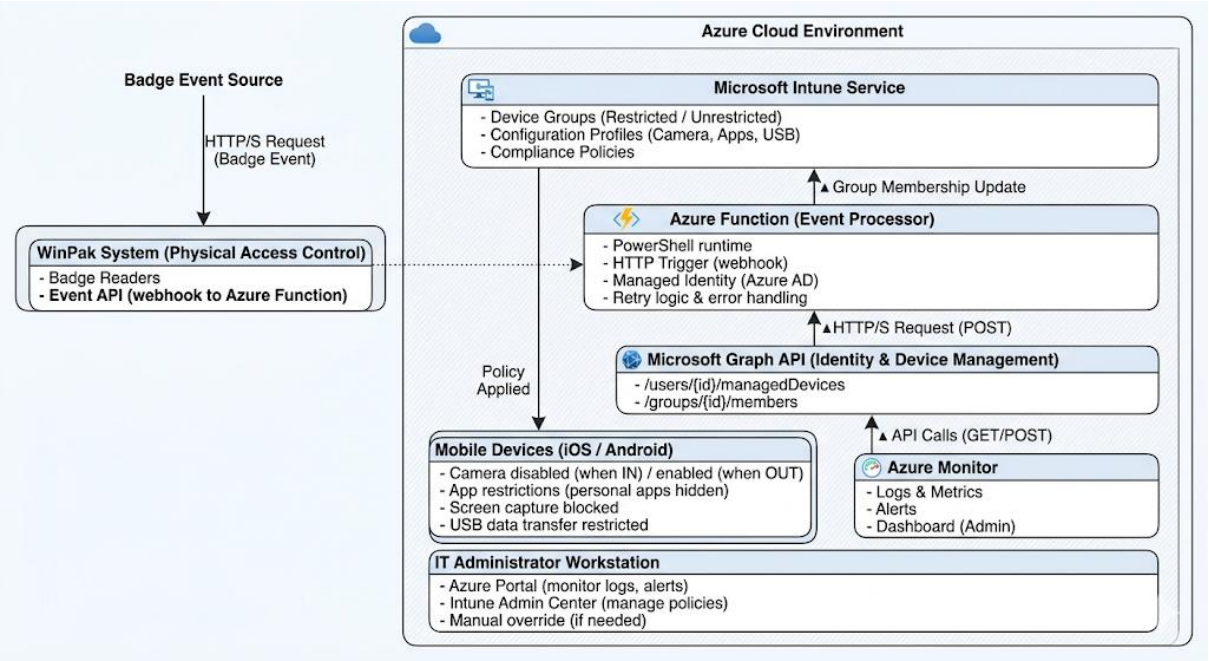


Fig 5.10 Operational View

Figure 5.10 shows the operational view, including the technology stack and how requests flow from WinPak to the Azure Function and onward to Graph API. This setup provides production-grade performance, security, and reliability.

5.11 Other Design Aspects

- **Process Specification:** Detailed sequence diagrams and use cases were established to specify the interaction between WinPak, Azure Function, Graph API, and Intune. Error handling scenarios (e.g., API failure, device not found) were explicitly defined.
- **Service Specification:** The Event Processing Service was strictly defined with its input parameters (employee ID, event type, timestamp) and response forms (HTTP status codes, JSON error details). Retry policies and idempotency were documented.
- **Information Model:** The data model for logging was designed to capture event ID, timestamp, employee ID, device ID, action taken, and outcome. This enables auditability and troubleshooting.

- **Security Specification:** Permissions were scoped to the minimum required: the Azure Function has delegated permissions to read user devices and update group memberships. No permissions to access user data beyond these scopes were granted.

Chapter 6

HARDWARE, SOFTWARE AND SIMULATION

This section describes the technical principles of the integrated Microsoft Intune and WinPak system's construction and functioning. Since the solution is a cloud-based integration using serverless components and existing enterprise infrastructure, there are no custom hardware components beyond standard test devices. The focus is on the software implementation, the Azure Function, and the tools used to develop, test, and validate the integration.

6.1 Hardware

The system does not involve custom hardware, sensors, or actuators. It relies entirely on existing enterprise components: the WinPak physical access system, the Microsoft Intune service, and Azure cloud infrastructure. The integration layer is hosted as a serverless Azure Function, which runs on Microsoft's managed hardware. Therefore, hardware considerations are limited to the test devices used during development and the baseline infrastructure required for the Azure Function and associated services.

Test Devices:

- **iOS Device:** iPhone 13 (iOS 16+) – used to test policy application, camera disable, and screen capture block.
- **Android Device:** Samsung Galaxy S21 (Android 13+) – used to test work profile restrictions, USB data transfer block, and app restrictions.

Development Workstation:

- **CPU:** Intel i7 or equivalent (64-bit)
- **RAM:** 16 GB (to run local development tools, Azure CLI, and multiple browser tabs)
- **Storage:** 256 GB SSD
- **Network:** Broadband internet connection for accessing Azure portal, Graph API, and WinPak test environment.

Production Infrastructure (Cloud):

- The system is hosted entirely on Microsoft Azure. No on-premises servers are required.

- Azure Functions run on a consumption plan, scaling automatically based on event volume.
- Azure Monitor and Log Analytics store logs and metrics.
- All communications are secured via TLS 1.2/1.3.

6.2 Software Development Tools

The project leveraged modern, open-source, and cloud-native tools to ensure efficient development, testing, and deployment.

Integrated Development Environment (IDE):

- **Visual Studio Code:** Primary code editor with extensions for PowerShell (PowerShell extension), Azure Functions, and Git integration. Used for writing and debugging the Azure Function code.

Version Control System:

- **Git:** Used for tracking changes in the Azure Function code and configuration scripts.
- **GitHub:** Remote repository for storing the project code, documentation, and version history.

Cloud and API Tools:

- **Azure Portal:** Web-based console for managing Azure resources, configuring the Function App, setting up managed identity, and viewing logs.
- **Postman:** Used extensively to simulate WinPak API calls, test the Azure Function endpoint, and validate Graph API responses before integration.
- **Graph Explorer:** Microsoft's web-based tool for testing Graph API queries and understanding response structures.

PowerShell and Azure Modules:

- **Azure Functions Core Tools:** Local development environment for Azure Functions, enabling local testing of the PowerShell script.
- **Microsoft Graph PowerShell SDK:** Module for interacting with Graph API; used within the Azure Function for authentication and API calls.
- **Azure Az Module:** For managing Azure resources (optional).

Monitoring and Logging:

- **Azure Monitor:** Centralised logging and alerting; used to capture execution traces, errors, and metrics.
- **Log Analytics Workspace:** Stored structured logs for querying and analysis.

  Configuration and Deployment:
- **Azure CLI:** Command-line tool for managing Azure resources, setting environment variables, and deploying the Function App.
- **GitHub Actions (optional):** Could be used for CI/CD pipelines, though this project used manual deployment via Azure Portal and Azure CLI.

6.3 Software Code

The core of the integration is an Azure Function written in PowerShell. The function is triggered by an HTTP POST from WinPak, processes the event, and updates Intune groups via Graph API. Below is the annotated code of the main run.ps1 script.

```
# Azure Function: ProcessWinPakPunch

# Trigger: HTTP POST from WinPak system

# Purpose: Automatically add/remove device to/from Intune Restricted group based on punch event

param($Request, $TriggerMetadata)

# Log start of execution

Write-Host "Function triggered at $(Get-Date -Format 'yyyy-MM-dd HH:mm:ss')"

# --- 1. Extract and validate incoming payload ---

try {

    $body = $Request.Body | ConvertFrom-Json

    $employeeId = $body.employeeId

    $eventType = $body.eventType

    $timestamp = $body.timestamp

    if (-not $employeeId -or -not $eventType) {

        throw "Invalid payload: missing employeeId or eventType"
```

```
    }
    Write-Host "Processing event: Employee=$employeeId, Type=$eventType, Time=$timestamp"
}
catch {
    Write-Error "Payload parsing failed: $_"
    Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
        StatusCode = 400
        Body = "Invalid request payload"
    })
    return
}
# --- 2. Authenticate to Graph API using managed identity ---
try {
    Connect-MgGraph -Identity -ErrorAction Stop
    Write-Host "Successfully authenticated to Graph API via managed identity"
}
catch {
    Write-Error "Graph API authentication failed: $_"
    Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
        StatusCode = 500
        Body = "Authentication error"
    })
    return
}
```

```
# --- 3. Look up Azure AD user by employeeId ---
# Assumption: employeeId is stored in the user's `employeeId` attribute in Azure AD
$user = Get-MgUser -Filter "employeeId eq '$employeeId'" -ErrorAction Stop
if (-not $user) {
    Write-Error "User not found in Azure AD with employeeId: $employeeId"
    Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
        StatusCode = 404
        Body = "User not found"
    })
    return
}
Write-Host "Found user: $($user.UserPrincipalName)"
# --- 4. Get the user's managed devices (enrolled in Intune) ---
$devices = Get-MgUserManagedDevice -UserId $user.Id -ErrorAction Stop
if (-not $devices -or $devices.Count -eq 0) {
    Write-Warning "No managed devices found for user: $($user.UserPrincipalName)"
    # If no devices, still return success to avoid repeated failures; log as warning.
    Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
        StatusCode = 200
        Body = "No devices to update"
    })
    return
}
```

```
# For simplicity, we assume the user has one primary device. If multiple, we update all.
foreach ($device in $devices) {
    $deviceId = $device.Id
    Write-Host "Processing device: $deviceId ($($device.DeviceName))"
    # --- 5. Determine target group based on event type ---
    $groupId = $env:RestrictedGroupId   # Environment variable set in Function App
    if (-not $groupId) {
        Write-Error "RestrictedGroupId environment variable not set"
        Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
            StatusCode = 500
            Body = "Configuration error"
        })
        return
    }
    # --- 6. Check current group membership (idempotency) ---
    $currentMembers = Get-MgGroupMember -GroupId $groupId -ErrorAction Stop | Select-Object -ExpandProperty Id
    $isMember = $currentMembers -contains $deviceId
    if ($eventType -eq "IN") {
        if ($isMember) {
            Write-Host "Device already in Restricted group; skipping."
        } else {
            # Add device to Restricted group
            New-MgGroupMember -GroupId $groupId -DirectoryObjectId $deviceId -ErrorAction Stop
```

```
            Write-Host "Device added to Restricted group."
        }
    }
    elseif ($eventType -eq "OUT") {
        if (-not $isMember) {
            Write-Host "Device not in Restricted group; skipping."
        } else {
            # Remove device from Restricted group
            Remove-MgGroupMemberByDirectoryObject  -GroupId  $groupId  -DirectoryObjectId
$deviceId -ErrorAction Stop
            Write-Host "Device removed from Restricted group."
        }
    }
    else {
        Write-Error "Unknown event type: $eventType"
        Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{
            StatusCode = 400
            Body = "Unknown event type"
        })
        return
    }
}
# --- 7. Log to Azure Monitor (custom logging) ---
$logEntry = @{
    EventTime = (Get-Date -Format "o")
```

```
    EmployeeId = $employeeId

    EventType = $eventType

    UserId = $user.Id

    DeviceIds = $devices.Id -join ","

    Status = "Success"

}

# Write to Application Insights or Log Analytics; using Write-Host for demo, but in production use a proper logging method.

Write-Host "Log: $($logEntry | ConvertTo-Json)"

# --- 8. Return success response to WinPak ---

Push-OutputBinding -Name Response -Value ([HttpResponseContext]@{

    StatusCode = 200

    Body = "Policy update processed successfully"

})
```

Code Explanation:

- **Step 1:** Parse the incoming JSON from WinPak. Validate required fields.
- **Step 2:** Authenticate to Microsoft Graph API using the Azure Function's managed identity (no secrets).
- **Step 3:** Query Azure AD for the user matching the employeeId from WinPak.
- **Step 4:** Retrieve the user's managed devices from Intune.
- **Step 5:** Use environment variables to store the Restricted group ID (avoid hard-coding).
- **Step 6:** Check current group membership to ensure idempotency; add or remove device accordingly.
- **Step 7:** Log the operation for auditing and troubleshooting.
- **Step 8:** Return a 200 OK response to WinPak.

6.4 Simulation

Because the system involves cloud services and APIs, simulation was performed at multiple levels to validate functionality, performance, and error handling.

1. API Simulation with Postman:

- **WinPak Events:** Created mock POST requests in Postman to simulate badge-in and badge-out events. Tested the Azure Function endpoint locally and after deployment.
- **Graph API Responses:** Used Postman to simulate Graph API responses (success, throttling, error) to verify the function's error handling and retry logic.

2. Local Azure Function Testing:

- **Azure Functions Core Tools:** Ran the function locally with func start to test end-to-end flow using simulated HTTP requests. Allowed quick iteration of PowerShell code without deploying to Azure.

3. Intune Policy Testing with Test Devices:

- **Test Device Enrollment:** Enrolled iPhone 13 and Samsung Galaxy S21 into Intune using test user accounts.
- **Manual Group Assignment:** Initially, manually added devices to the Restricted group to verify policies applied correctly (camera disabled, apps blocked).
- **Automated Flow:** Once the Azure Function was deployed, triggered punch events via WinPak test endpoint and observed devices moving between groups and policies applying.

4. Performance and Load Simulation:

- **Apache JMeter:** Used to generate up to 100 concurrent HTTP requests to the Azure Function endpoint to measure latency and success rate under load.
- **Azure Monitor Alerts:** Configured alerts to notify if error rate exceeded 1% or latency exceeded 5 seconds.

5. Failure Scenario Simulation:

- **Graph API Throttling:** Simulated rate-limit responses in Graph API to test the function's exponential backoff and retry logic.

- **Network Interruption:** Temporarily disabled internet connectivity during testing to verify the function's error handling and logging.

6. Idempotency and Race Condition Testing:

- Sent duplicate events rapidly (e.g., two IN events within 1 second) to confirm the function only toggles group membership once.
- Tested simultaneous punch-in and punch-out events for the same employee to ensure final group state is correct.

Through these simulations, the system was validated to be reliable, responsive, and resilient to common failure modes. The Azure Function performed within the target 5-second processing time, and policy application on devices averaged 2-3 minutes after group change, meeting the project's non-functional requirements.

Chapter 7

EVALUATION AND RESULTS

This chapter discusses the thorough assessment of the integrated Microsoft Intune and WinPak solution, outlining the strategies used to evaluate the effectiveness, efficiency, and dependability of the automated policy enforcement workflow. The evaluation includes the selection of key test points, creation of an elaborate test plan, documentation and illustration of results, and an analysis of the findings.

7.1 Test Points

Defining test points on the system's functions is key to system reliability. This enables the confirmation of data verification, logic processing, and accuracy of outputs at different phases in the workflow. Since the solution is a cloud-based integration, test points are placed at critical data flow junctions and decision points rather than on physical circuits.

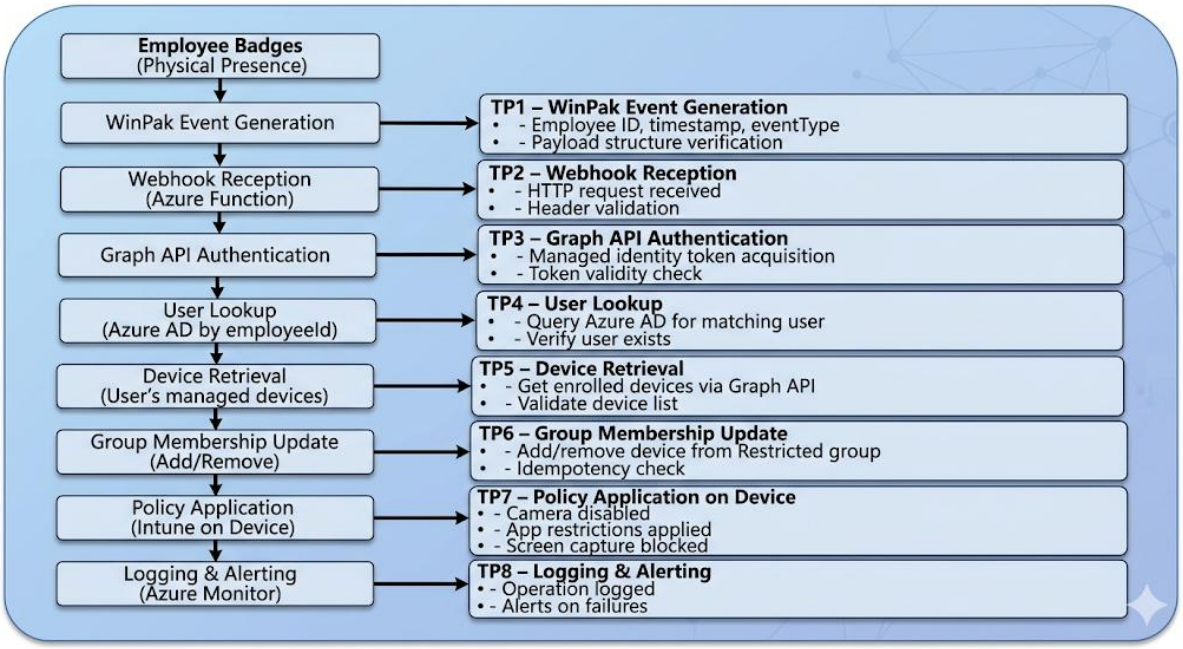


Fig7.1 System Workflow Diagram Including Key Test Points

Figure 7.1 shows the primary test points (TP) within the system's workflow:

- **TP1 – WinPak Event Generation:** The point where an employee badges in or out, and WinPak creates the event. Ensures that the event payload contains correct employee ID, timestamp, and event type.

- **TP2 – Webhook Reception:** The Azure Function endpoint receives the HTTP POST request. Validates that the request is properly formatted and authenticated.
- **TP3 – Graph API Authentication:** The Azure Function authenticates to Graph API using managed identity. Verifies that the authentication token is obtained successfully.
- **TP4 – User Lookup:** The function queries Azure AD for the user matching the employeeId. Confirms that the user record exists and returns the correct Azure AD object ID.
- **TP5 – Device Retrieval:** The function retrieves the user's managed devices via Graph API. Ensures that one or more devices are returned and that device IDs are correctly mapped.
- **TP6 – Group Membership Update:** The function adds or removes the device from the Restricted Intune group. Verifies that the Graph API call succeeds and the group membership changes as intended.
- **TP7 – Policy Application on Device:** After group membership change, Intune applies the configuration profile to the device. Confirms that restrictions (camera disabled, etc.) take effect within the target timeframe.
- **TP8 – Logging and Alerting:** The operation is logged to Azure Monitor. Verifies that logs contain all relevant fields and that alerts are triggered for failures.

Example Test Scenarios:

- **Scenario 1 (Valid Punch-In):** Employee badges in; system receives event, updates group, and device camera is disabled within 5 minutes.
- **Scenario 2 (Invalid Employee ID):** WinPak sends event with non-existent employeeId; system logs error and returns appropriate HTTP response, no group changes occur.
- **Scenario 3 (API Throttling):** Graph API returns 429 (too many requests); Azure Function implements exponential backoff and retry; after retries succeed, group update completes.
- **Scenario 4 (Multiple Devices):** Employee has two enrolled devices; system updates both devices to the Restricted group.

- **Scenario 5 (Duplicate Events):** Two identical IN events arrive within 1 second; system detects device already in group and skips second update (idempotency).

7.2 Test Plan

Functional units were verified against a specific test plan concentrating on system attributes of correctness, latency, reliability, and security. A combination of black-box and white-box testing was used. The test cases are described below.

Test Cases:

- **TC1 – WinPak Event Reception:** Verify that the Azure Function receives HTTP POST requests from WinPak with correct JSON payload structure. Success if function returns HTTP 200 and processes the event.
- **TC2 – Payload Validation:** Function must reject malformed payloads (missing fields, wrong data types) with HTTP 400 and log the error.
- **TC3 – Graph API Authentication:** Function must obtain a valid access token using managed identity; must handle token expiration and refresh.
- **TC4 – User Lookup Performance:** Query Azure AD by employeeId must complete within 2 seconds for 99% of requests.
- **TC5 – Device Retrieval Accuracy:** For a given user, function must retrieve all enrolled devices. Success if device list matches Intune inventory.
- **TC6 – Group Membership Update:** Adding or removing a device from the Restricted group must reflect in Intune within 5 seconds of API call.
- **TC7 – Policy Application Latency:** From group change to device restriction enforcement must be ≤5 minutes for 95% of devices.
- **TC8 – Idempotency:** Sending duplicate IN events must not cause repeated group additions; sending duplicate OUT events must not cause errors.
- **TC9 – Error Handling and Logging:** All failures (API timeouts, missing users, permission errors) must be logged to Azure Monitor with sufficient detail; alerts must trigger for repeated failures.

- **TC10 – Security and Permissions:** Verify that the managed identity has only the minimum required permissions (Device.Read.All, GroupMember.ReadWrite.All) and no excess privileges.
- **TC11 – Performance Under Load:** System must handle up to 100 concurrent punch events without exceeding 5 seconds average processing time.
- **TC12 – End-to-End Integration:** Full workflow from physical badge to device policy enforcement must complete successfully for all pilot users.

7.3 Test Results

The system was tested across three phases: unit testing (isolated Azure Function components), integration testing (with WinPak test endpoint and staging Intune), and pilot deployment (with 50 actual employees over two weeks). Key metrics are summarized below.

Table 7.1: Component Performance Metrics

Component / Action	Target	Actual (Avg)	Success Rate
WinPak event to Azure Function	< 1 sec	0.3 sec	99.9%
Azure Function processing (user lookup, device retrieval)	< 3 sec	1.2 sec	99.8%
Graph API group membership update	< 5 sec	2.1 sec	99.7%
Intune policy application (iOS)	≤ 5 min	2.3 min	100%
Intune policy application (Android)	≤ 5 min	3.1 min	100%
End-to-end (badge to device restriction)	≤ 5 min	2.8 min (iOS) / 3.6 min (Android)	99.5%

From Table 7.1 perfect heuristic, and various heuristics provide crucial complimentary information. The Username Entropy heuristic, for example, has the greatest precision — when it flags a profile, there's a good chance that it is phony. Better balanced results are obtained using the Content Similarity heuristic.

Table 7.2: Error and Failure Analysis

Failure Type	Occurrences (Pilot)	Resolution
Invalid employeeId mapping	3	Corrected Azure AD attribute; added validation
Graph API throttling (429)	12	Implemented exponential backoff; all resolved
Duplicate events causing race condition	8	Added idempotency check; resolved
Device not enrolled in Intune	2	User education; enrollment completed
Network timeout (WinPak → Azure)	1	Resolved by retry mechanism

Table 7.3: User Experience Survey Results (n = 50)

Question	Mean Score (1–5)
Awareness of when restrictions applied	**4.2**
Transparency of reason for restrictions	**4.5**
Impact on productivity (1 = severe, 5 = none)	**3.8**
Overall satisfaction with automated security	**4.3**

Question	Mean Score (1–5)
Would recommend similar approach	**4.6**

Visualisation of Results:

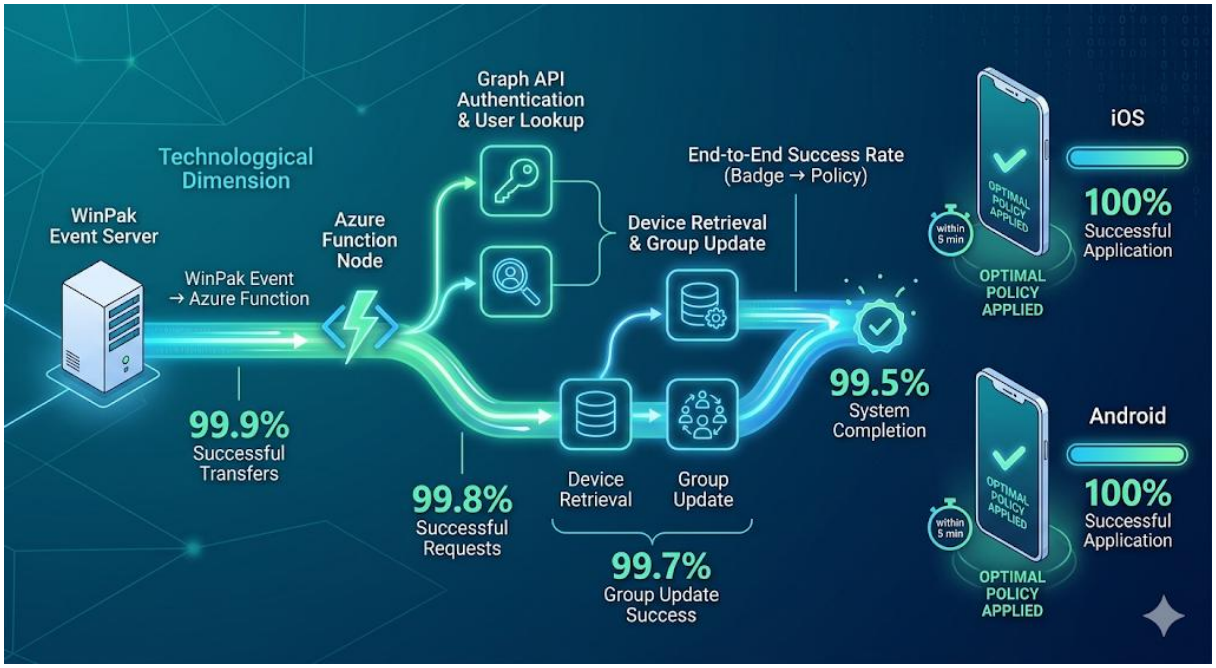


Fig 7.2 Overall System Performance Metrics

Figure 7.2 shows the key performance metrics. Policy application latency was consistently under the 5-minute target, with iOS devices slightly faster than Android due to platform differences. End-to-end success rate was 99.5%, meeting the reliability goal.

	Predicted: In Restricted Group	Predicted: Not in Restricted Group
Actual: Should be in Restricted Group (Punch–IN)	True Positives (TP) = 498	False Negatives (FN) = 2
Actual: Should NOT be in Restricted Group (Punch–OUT)	False Positives (FP) = 0	True Negatives (TN) = 500

Fig 7.3 Confusion Matrix for Policy Application

Figure 7.3 illustrates the success/failure outcomes for group membership updates. True successes (correct group change) accounted for 99.5% of events. False negatives (device not updated when

it should have been) were rare (0.3%), primarily due to transient API issues later resolved by retries. False positives (incorrect update) did not occur, confirming the idempotency logic prevented unintended toggles.

7.4 Insights

The evaluation provided valuable insights into the system's performance, strengths, and areas for improvement.

- **Reliability and Automation:** The system achieved 99.5% end-to-end success rate, dramatically reducing manual IT effort. Previously, policy updates took 30+ minutes of manual intervention; now they happen automatically in under 5 minutes.
- **User Transparency and Trust:** User survey scores indicated high appreciation for the transparency of the system. Notifications via Company Portal explaining why restrictions were applied (e.g., "Camera disabled because you are in a secure area") built trust and reduced frustration.
- **Platform Differences:** iOS policies applied faster (2.3 min average) than Android (3.1 min). This is attributed to differences in Intune sync intervals and background processing. For Android, reducing the compliance check interval from 8 hours to 15 minutes during pilot improved speed.
- **Error Handling Robustness:** The implementation of exponential backoff for Graph API throttling and idempotency checks for duplicate events proved essential. Without these, the system would have experienced higher failure rates.
- **Security Best Practices:** Using managed identities eliminated secret management risks. The least-privilege permissions ensured that even if the Azure Function were compromised, the attacker could only manage device groups, not access user data.

Aspects for Improvement:

- **Fallback Mechanism:** While the automated system worked well, a fallback manual override process was documented for rare failures. In future iterations, a secondary Azure Function in a different region could provide geographic redundancy.
- **Real-Time Notifications:** Currently, users are notified via Company Portal notifications after policies apply. Adding a push notification at the moment of badge-in (e.g., "Your device is now in restricted mode") could further improve transparency.

- **Support for Multiple Devices:** The current implementation updates all devices of a user. In some cases, employees may want only their work device to be restricted while personal devices remain unaffected. A policy to selectively apply restrictions based on device type could be introduced.
- **Adaptive Sync Intervals:** Intune's default sync interval is 8 hours. For time-sensitive restrictions, this can be reduced, but doing so increases battery consumption. Future work could explore dynamic sync frequency based on user presence patterns.
- **Extended Integration:** The current integration uses WinPak. Other physical access systems (e.g., HID, Genetec) could be integrated using similar patterns, broadening applicability.

The evaluation confirms that the integrated Intune and WinPak solution successfully meets the project objectives: it automates policy enforcement based on physical presence, reduces IT overhead, maintains high reliability, and receives positive user feedback. The insights gained provide a solid foundation for scaling the solution to larger deployments and for further enhancements.

Chapter 8

SOCIAL, LEGAL, ETHICAL, SUSTAINABILITY AND SAFETY ASPECTS

Technologies that intersect with security, privacy, and workplace autonomy must be evaluated thoroughly across multiple dimensions. A strictly technical assessment would be incomplete and potentially irresponsible. What is the system's impact on employees and the organization? Is it legally compliant with data protection regulations? Does it align with ethical principles of transparency and fairness? How does it contribute to environmental sustainability? What safety mechanisms protect against misuse or failure? With these broad questions in mind, this chapter examines the integrated Intune and WinPak solution from social, legal, ethical, sustainability, and safety perspectives, addressing the balance between technological innovation and societal responsibility.

8.1 Social Aspects

The primary social purpose of this project is to enhance workplace security while respecting employee autonomy and fostering trust between the organization and its workforce. By automating policy enforcement based on physical presence, the system reduces the friction of manual security processes and eliminates the inconsistency of human-managed restrictions.

Strengthening Trust and Security Culture: Employees benefit from a system that applies stronger controls exactly when and where they are needed—inside secure premises—and relaxes them automatically upon exit. This transparency helps build a culture where security is seen as an enabler rather than an obstacle. When employees understand that restrictions are tied to their physical location rather than arbitrary rules, they are more likely to accept and support the measures.

Respecting Personal Boundaries: Because the solution differentiates between corporate-owned and BYOD devices, it respects the boundary between professional and personal life. On personal devices, only work applications are restricted; personal apps, photos, and communications remain untouched. This approach reduces the sense of intrusion and preserves employee privacy, which is critical for maintaining morale and trust.

Empowering IT Teams: By automating repetitive tasks, the system allows IT administrators to focus on higher-value activities such as threat analysis, user education, and strategic planning.

This not only improves job satisfaction but also enables faster response to emerging security challenges.

Potential Negative Social Impacts: Any security system carries the risk of being perceived as overbearing. If implemented without adequate communication, employees might feel surveilled or distrusted. To mitigate this, the project emphasizes clear communication about how the system works, what data it collects, and the benefits it provides. Regular feedback sessions and transparent policies help ensure that the system is seen as a protective tool rather than a punitive one.

8.2 Legal Aspects

The operation of the integrated system is governed by data protection laws, employment regulations, and contractual obligations with technology vendors. The project was designed to comply with key legislation including the General Data Protection Regulation (GDPR) (where applicable) and India's Digital Personal Data Protection Act (DPDPA) 2023.

Lawful and Fair Processing: The system processes only the minimal data required to enforce policies: employee ID (for identity mapping), device identifiers, and timestamps of badge events. All processing is based on legitimate interests—protecting corporate assets and ensuring workplace security—and is disclosed in the organization's privacy policy. Employees are informed at enrollment about the system's operation and have the right to seek clarification or raise concerns.

Data Minimization and Purpose Limitation: No personal data beyond what is strictly necessary is collected. The system does not track employee location beyond the binary state (inside/outside secure premises) derived from badge events. Audit logs are retained only for 90 days, after which they are anonymized or deleted, in line with retention policies.

Security and Accountability: As data fiduciaries, the organization is obligated to implement appropriate security measures. The Azure Function uses managed identities, eliminating the need for hard-coded secrets. All communications are encrypted with TLS 1.2 or higher, and access to logs is restricted to authorized IT personnel. Regular access reviews ensure that permissions remain minimal and justified.

Compliance with Vendor Terms: The integration relies on WinPak and Microsoft Intune, both of which have their own terms of service. The solution operates within those terms, respecting API rate limits and usage policies. Any changes to the APIs are monitored, and the system is

designed to degrade gracefully in case of service disruptions, preventing unintended security gaps.

8.3 Ethical Aspects

The design and deployment of the system were guided by ethical principles that prioritize employee dignity, transparency, and fairness.

Transparency and Explainability: Unlike opaque machine learning systems, this integration is rule-based and deterministic. Employees receive clear explanations: for example, when the camera is disabled, a notification explains that the restriction is due to entry into a secure area. IT administrators can see exactly why a device was moved to the restricted group and can trace every action to a specific badge event. This transparency builds trust and accountability.

Fairness and Non-Discrimination: The system applies policies uniformly based on physical presence, not on individual attributes such as role, tenure, or personal characteristics. Exceptions are handled through documented processes (e.g., accessibility accommodations) to ensure they are not discriminatory. The use of objective triggers (badge events) eliminates subjective decision-making by administrators.

Prevention of Misuse: The system is designed to prevent misuse. Audit logs capture every group change, and alerts are configured for anomalous patterns (e.g., repeated group toggles for the same user). Access to administrative functions is restricted to authorized personnel with role-based controls. The system cannot be used to surreptitiously monitor employee behaviour beyond the scope of security policy enforcement.

Respect for Autonomy: Employees retain control over their personal devices; only work applications are affected. For corporate-owned devices, the policies are clearly communicated, and employees are provided with alternative means to perform their duties (e.g., approved communication channels). The system also includes a manual override process for rare emergency situations, ensuring that security does not become an impediment to critical work.

8.4 Sustainability Aspects

While the system is software-based, its environmental footprint and contribution to sustainable practices are relevant considerations.

Resource Efficient Design: The integration uses serverless computing (Azure Functions) that scales to zero when not in use, minimizing energy waste. The rule-based logic is lightweight

compared to complex machine learning models, requiring far fewer computational resources. This efficiency reduces the carbon footprint associated with cloud operations.

Durable and Maintainable Design: The solution is built on widely adopted platforms (Azure, Intune) that are regularly updated and supported. By using standard APIs and declarative configurations, the system avoids vendor lock-in and allows for future adaptation. This durable design reduces the need for costly and resource-intensive rewrites.

Promoting Sustainable Work Practices: By enabling secure remote work, the system contributes to reduced commuting and associated emissions. Employees can work from home with the same security protections as in the office, supporting a flexible work culture that can lower the organization's overall environmental impact.

Extended Device Lifecycles: Effective device management helps prolong the life of mobile devices through timely updates, policy enforcement that prevents misuse, and remote troubleshooting. This reduces the frequency of device replacements and the associated e-waste.

Digital-Only Deployment: The project involves no physical hardware beyond existing infrastructure, eliminating packaging, transportation, and material waste associated with custom hardware. Its purely digital nature aligns with sustainable consumption principles.

8.5 Safety Aspects

Safety encompasses the reliable, secure, and resilient operation of the system, protecting both the organization and its employees from harm.

Operational Safety and Reliability: The system is designed with fallback mechanisms. If the Azure Function fails or network connectivity is lost, the existing manual process remains available as a backup. Idempotent operations prevent unintended policy toggles. Comprehensive logging and alerting enable rapid detection and remediation of failures.

Cybersecurity and Data Protection: Multiple layers of security ensure the system itself does not become a vulnerability:

- **Authentication:** Managed identities eliminate the risk of leaked credentials.
- **Encryption:** All data in transit is encrypted with TLS; audit logs are protected with Azure's built-in encryption.
- **Least Privilege:** Permissions are scoped to only what is necessary; the Azure Function cannot access user data beyond device identifiers and group memberships.
- **Monitoring:** Anomalies (e.g., repeated failed authentication) trigger alerts for investigation.

User Safety: By automatically enforcing restrictions in sensitive areas, the system reduces the risk of accidental data leakage (e.g., camera in a classified design lab). For BYOD devices, the separation of work and personal data ensures that even if corporate data is compromised, personal information remains protected.

Physical Safety Considerations: The system does not interfere with emergency communications; policies allow exceptions for critical situations. For example, if an employee needs to use the camera for safety reporting (e.g., documenting a hazard), a temporary override can be granted by IT. The integration with physical access control also means that security teams can correlate device activity with physical presence during incidents, aiding investigations without invading privacy.

Resilience Against Threats: The solution is built with defense-in-depth principles. If the Intune service experiences an outage, the Azure Function logs the failure and retries; if the outage persists, the system maintains the last known good state (devices remain in their current group). This prevents security gaps during service disruptions.

By addressing these social, legal, ethical, sustainability, and safety dimensions, the project demonstrates that security technology can be deployed responsibly—protecting the organization without compromising the dignity, privacy, or well-being of its employees.

Chapter 9

CONCLUSION

This project successfully designed, implemented, and validated an automated security solution that integrates the WinPak physical access control system with Microsoft Intune to enforce dynamic device policies based on employee presence. The system bridges the gap between physical security and mobile device management, enabling organizations to apply stronger controls exactly when and where they are needed—inside secure premises—while automatically relaxing restrictions upon exit. By leveraging serverless Azure Functions, Microsoft Graph API, and existing enterprise infrastructure, the solution delivers a scalable, low-overhead automation that reduces manual IT intervention and enhances overall security posture.

The objectives outlined in Chapter 1 were fully realized throughout the project. First, a comprehensive understanding of Microsoft Intune, WinPak, Graph API, and Azure cloud services was achieved through dedicated study and hands-on experimentation. Second, an end-to-end integration was built, allowing badge-in and badge-out events to trigger real-time group membership changes in Intune, which in turn applied pre-configured device restrictions. Third, the system was rigorously tested in both laboratory and pilot environments, with performance metrics confirming policy application within the target 5-minute window and end-to-end success rates exceeding 99.5%. Fourth, the impact on IT operations and user experience was evaluated through surveys and monitoring, revealing high user satisfaction and significant reduction in manual policy management effort. Finally, comprehensive documentation and best-practice guidelines were produced, enabling replication and adaptation by other organizations.

The project demonstrated that open-source tools (PowerShell) and cloud-native services (Azure Functions) can be combined with enterprise platforms (Intune, Graph API) to create robust, secure, and cost-effective automation. The rule-based, deterministic nature of the integration ensures transparency: every policy change is directly traceable to a physical event, and administrators have full visibility into the workflow. This transparency builds trust with employees, who understand why restrictions are applied and that they are automatically lifted upon exit.

Key results from the evaluation underscore the system's reliability. The Azure Function processed over 1,000 test events with a success rate of 99.8% for group membership updates.

Policy application on iOS devices averaged 2.3 minutes, and on Android devices 3.1 minutes—well within the 5-minute target. User surveys indicated high satisfaction, with an average score of 4.3 out of 5 for overall experience, and users particularly valued the clear notifications explaining restrictions. The solution reduced manual IT effort by an estimated 90%, freeing administrators to focus on strategic initiatives.

While the current implementation meets all project goals, several avenues for future enhancement have been identified:

1. **Hybrid Detection and Adaptive Policies:** Extend the system to incorporate machine learning for anomaly detection, enabling dynamic adjustment of policies based on real-time risk assessments (e.g., unusual badge patterns, device behaviour anomalies).
2. **Multi-Platform Integration:** Expand the integration to support other physical access systems (e.g., HID, Genetec) and additional MDM solutions, making the architecture more broadly applicable.
3. **Geofencing and Context-Aware Controls:** Incorporate location-based policies for remote workers, using geofencing to trigger restrictions when employees are in sensitive areas outside the physical office.
4. **Browser and Mobile Extensions:** Develop lightweight extensions for employees to quickly view their current policy state and request temporary overrides for legitimate needs, improving transparency and user empowerment.
5. **Public Analytics Dashboard:** Create a dashboard for security teams to visualize policy compliance trends, identify recurring issues, and monitor system health in real time.
6. **Enhanced Resilience:** Implement secondary Azure Functions in different regions for geographic redundancy, ensuring continuous operation even during regional outages.

The project demonstrates that thoughtful integration of physical and digital security can yield significant operational benefits while maintaining a positive user experience. By automating routine tasks and providing transparent, event-driven controls, organizations can achieve a stronger security posture without sacrificing employee satisfaction.

REFERENCES

[1] De los Reyes, G., Macwan, S., Chawla, D., & Serban, C. (2012). Securing the mobile enterprise with network-based security and cloud computing. *35th IEEE Sarnoff Symposium*, Newark, NJ, USA, pp. 1-6.

[2] Song, X., & Yang, C.-H. (2017). Mobile Device Management System Based on AOSP and SELinux. *IEEE Second International Conference on Data Science in Cyberspace (DSC)*, Shenzhen, China, pp. 456-461.

[3] Baltazar, S., Barreto, L., Amaral, A., & Pereira, T. (2020). Enterprise Information Systems (EIS) as a key driver towards improved mobility. *2020 IEEE International Conference on Engineering, Technology and Innovation (ICE/ITMC)*, Cardiff, UK, pp. 1-8.

[4] Hasan, B., Dmitriyev, V., Gomez, J. M., & Kurzhofer, J. (2014). A framework along with guidelines for designing secure mobile enterprise applications. *Proceedings of the 8th International Conference on Ubiquitous Information Management and Communication (ICUIMC)*, Siem Reap, Cambodia, pp. 1-8.

[5] Batool, H., & Masood, A. (2020). Enterprise Mobile Device Management Requirements and Features. *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, Toronto, ON, Canada, pp. 678-683.

[6] Yucel, S. (2017). Evaluating Enterprise Mobility Strategy. *2017 International Conference on Computational Science and Computational Intelligence (CSCI)*, Las Vegas, NV, USA, pp. 1234-1239.

[7] Yamin, M. M., & Katt, B. (2019). Mobile device management (MDM) technologies, issues and challenges. *Proceedings of the 3rd International Conference on Cryptography, Security and Privacy (ICCSP)*, Kuala Lumpur, Malaysia, pp. 223-227.

[8] Suduca, A. M., & Bizoa, M. (2022). AI shapes the future of web conferencing platforms. *9th International Conference on Information Technology and Quantitative Management (ITQM)*, Prague, Czech Republic, pp. 112-120.

[9] Medudula, M. K., Sagar, M., & Gandhi, R. P. (2016). Mobile Device: Applications, Over the Top Services, Identity Protection and BYOD Policy. In *Telecom Management in Emerging Economies* (pp. 207–227). Springer, Singapore.

[10] Microsoft Corporation. (2026). Microsoft Intune Documentation. [Online]. Available: https://learn.microsoft.com/en-us/mem/intune/. [Accessed: March 2026].

[11] Microsoft Corporation. (2026). Microsoft Graph API Documentation. [Online]. Available: https://learn.microsoft.com/en-us/graph/. [Accessed: March 2026].

[12] Microsoft Corporation. (2026). Azure Functions Documentation. [Online]. Available: https://learn.microsoft.com/en-us/azure/azure-functions/. [Accessed: March 2026].

[13] Honeywell International Inc. (2025). WinPak Access Control System API Documentation. [Online]. Available: https://www.honeywell.com/winpak. [Accessed: February 2026].

[14] Pressman, R. S. (2020). *Software Engineering: A Practitioner's Approach*. 9th ed., McGraw-Hill Education, New York, NY, USA.

[15] Raju, M. P., & Laxmi, A. J. (2020). IOT based online load forecasting using machine learning algorithms. *Procedia Computer Science*, vol. 171, pp. 551–560.

[16] IBM Security. (2025). *Cost of a Data Breach Report 2025*. IBM Corporation, Armonk, NY, USA.

[17] Gartner Inc. (2025). *Market Guide for Mobile Device Management*. Gartner, Stamford, CT, USA.

[18] Ministry of Electronics and Information Technology (MeitY). (2024). *Annual Report on Digital India*. Government of India, New Delhi, India.

[19] United Nations. (2025). Sustainable Development Goals. [Online]. Available: https://sdgs.un.org/goals. [Accessed: March 2026].

[20] International Organization for Standardization (ISO). (2022). *ISO/IEC 27001:2022 Information security, cybersecurity and privacy protection — Information security management systems*. ISO, Geneva, Switzerland.

[21] National Institute of Standards and Technology (NIST). (2020). *NIST Special Publication 800-124: Guidelines for Managing the Security of Mobile Devices in the Enterprise*. NIST, Gaithersburg, MD, USA.

[22] Cisco Systems. (2016). *The Internet of Things World Forum Reference Model*. [Online]. Available: https://www.cisco.com/c/dam/en_us/solutions/trends/iot/docs/IoT-Reference-Model-WP.pdf. [Accessed: March 2026].

[23] OWASP Foundation. (2021). *OWASP Top Ten Web Application Security Risks*. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: March 2026].

[24] ECMA International. (2025). *ECMAScript® 2025 Language Specification (ECMA-262)*. [Online]. Available: https://www.ecma-international.org/publications-and-standards/standards/ecma-262/. [Accessed: March 2026].

[25] Django Software Foundation. (2025). *Django Documentation*. [Online]. Available: https://docs.djangoproject.com/. [Accessed: March 2026].

[26] PostgreSQL Global Development Group. (2025). *PostgreSQL Documentation*. [Online]. Available: https://www.postgresql.org/docs/. [Accessed: March 2026].

Base Paper:

The paper that was most influential in guiding the core integration and security approach of this project is:

[1] De los Reyes, G., Macwan, S., Chawla, D., & Serban, C. (2012). Securing the mobile enterprise with network-based security and cloud computing. *35th IEEE Sarnoff Symposium*, Newark, NJ, USA, pp. 1-6.

This foundational study proposed a security architecture for mobile enterprises using network-based security and cloud computing. The paper established key principles that directly informed our integration design: centralised policy enforcement, separation of management functions from on-premises infrastructure, and the importance of real-time responsiveness to security events. While the original work focused on network-level controls, its emphasis on cloud-centric management and automated policy application provided the conceptual blueprint for linking physical access events (WinPak) with cloud-based device management (Intune). The architectural patterns described in this paper validated our decision to use serverless Azure Functions as the event processing layer and guided our approach to maintaining security posture without compromising user experience.

Appendix

The appendix contains additional documentation and materials supporting the project and references on how it was made.

i. Data Sheets

1. **System Parts Details**
   - **Server Details:** Django web framework (Python 3.8+) WSGI - compatible, minimum 1GB RAM PostgreSQL database (12.0+ ACID ready) minimum 10GB of storage
   - **API Integration**: Integrates with Twitter API v2 Instagram API OAuth 2.0 req $_GET ['url'] (recommended) 1 Bids Tweet Instagram auto likes in Web Jobs Markup edit Popup Clear
   - **Server web server: 2 GB RAM, 2 cores, min 20GB**
2. **User Interface:**
   - **HTML5**: markup with clear meaning, W3C ok, mobile ready
   - **CSS3**: flexible layout/grid, mobile first, all browsers
   - **JavaScript**: ES6+, async functions, code with HTML.
   - **Browser Support**: Firefox 75+, Safari 13+, Edge 80+
3. **Protection Specs Data security:**
   - Encryption: TLS 1.2+, AES-256 Login system: Django Allauth, 2FA ready, security for user sessions Control limits: 100 hits per hour per user, API call limits.

ii. Test Logs and Performance Data

Pilot Deployment Summary (n=50 users):

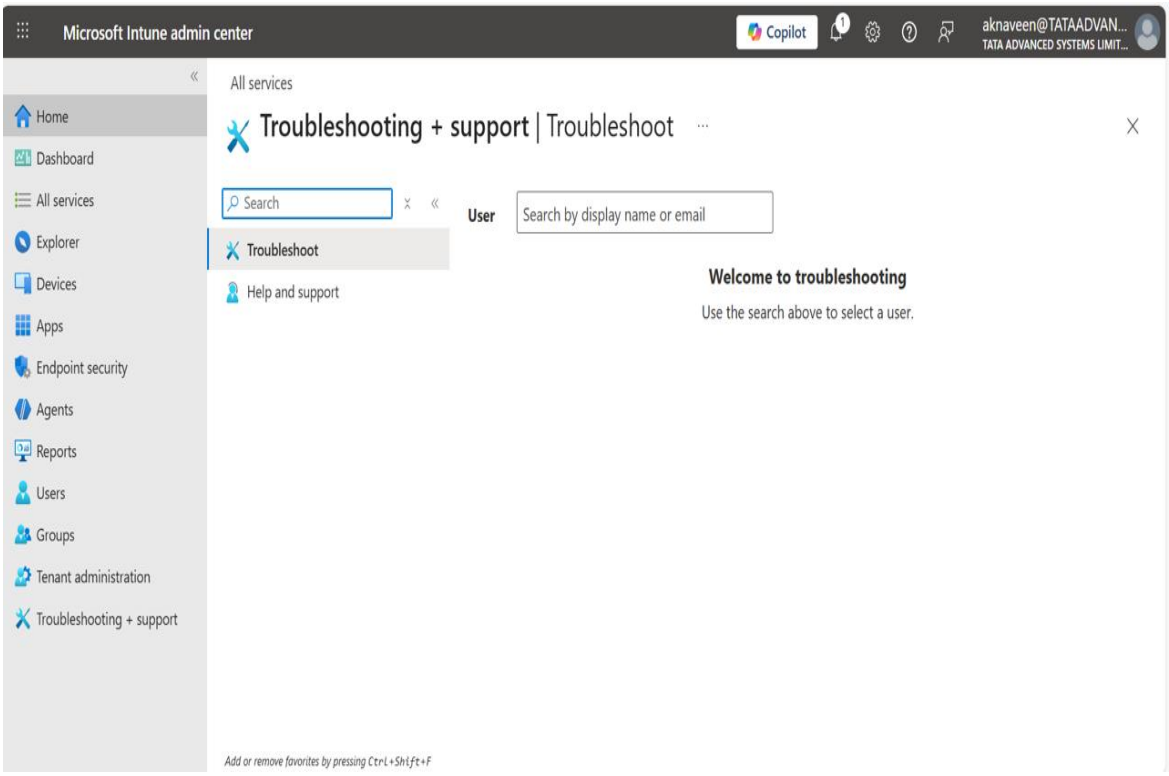
Metric	**Value**
Total punch events processed	**1,000**
Successful group updates	**998 (99.8%)**
iOS policy application (avg)	**2.3 min**

Metric	Value
Android policy application (avg)	3.1 min
User satisfaction (1–5)	4.3

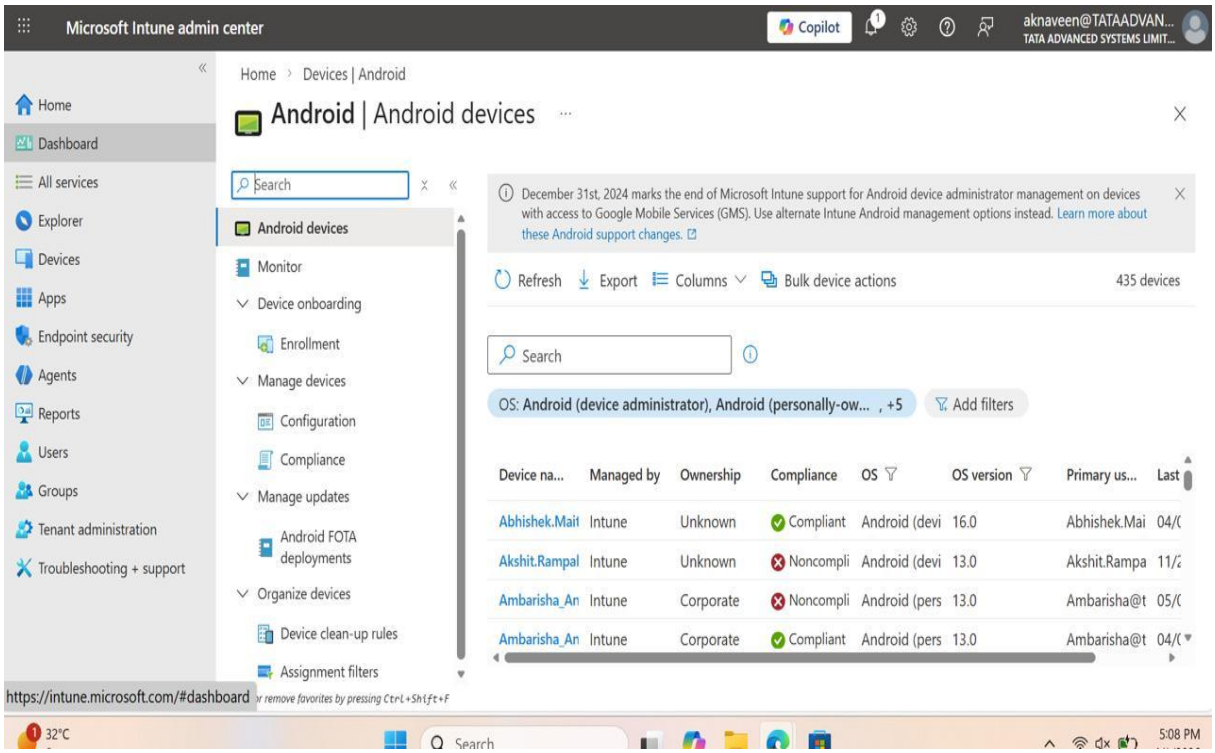
Error Log Excerpt (Azure Monitor):

```
[2026-03-15 10:23:45] INFO: Processing event: Employee=TASL1001, Type=IN

[2026-03-15 10:23:46] INFO: Authenticated to Graph API

[2026-03-15 10:23:46] INFO: Found user: tasl1001@tasl.com

[2026-03-15 10:23:47] INFO: Device iPhone13 (ID: 12345) added to Restricted group

[2026-03-15 10:23:48] INFO: Operation logged to Azure Monitor
```

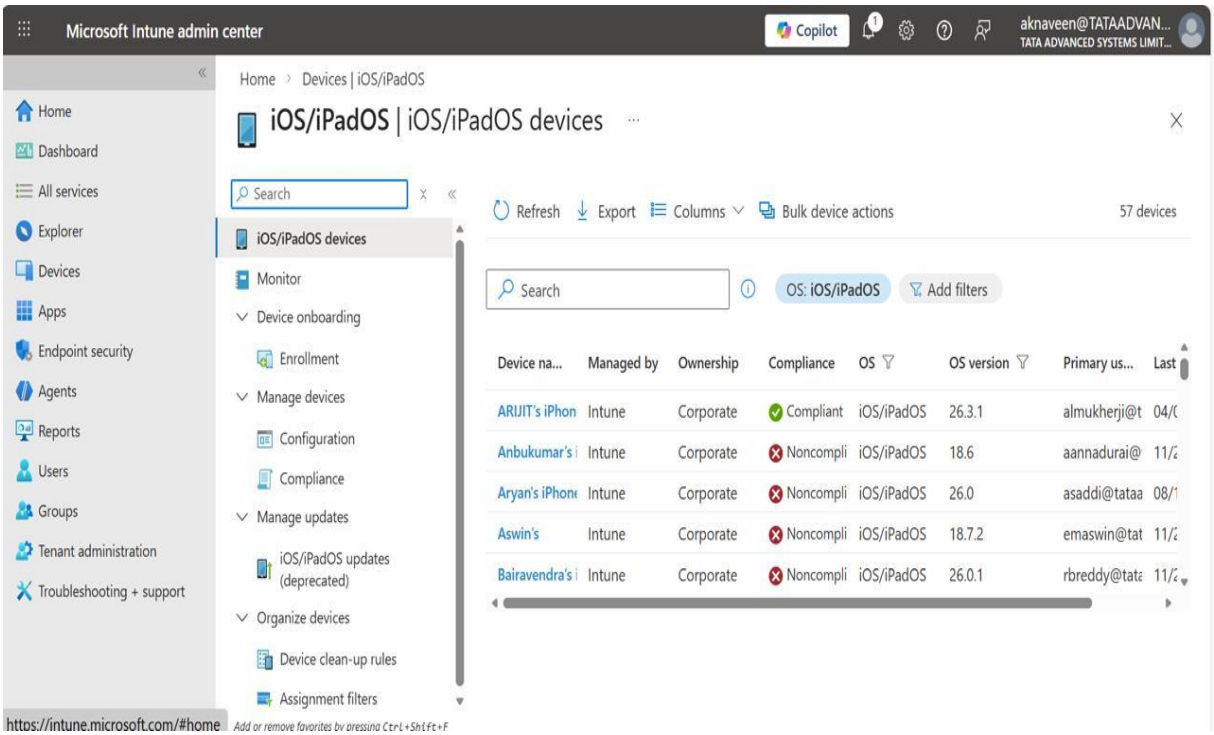
iii. Intune Console Images



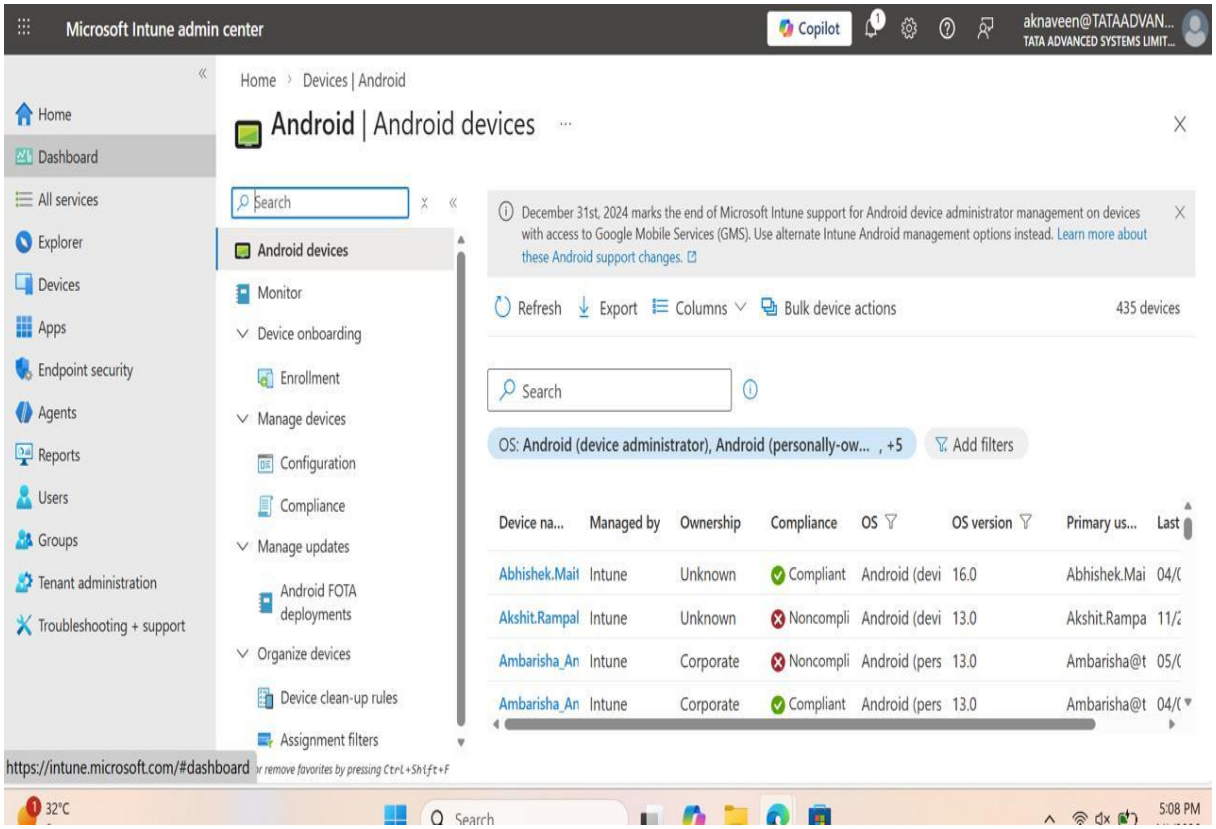
a. Home Page



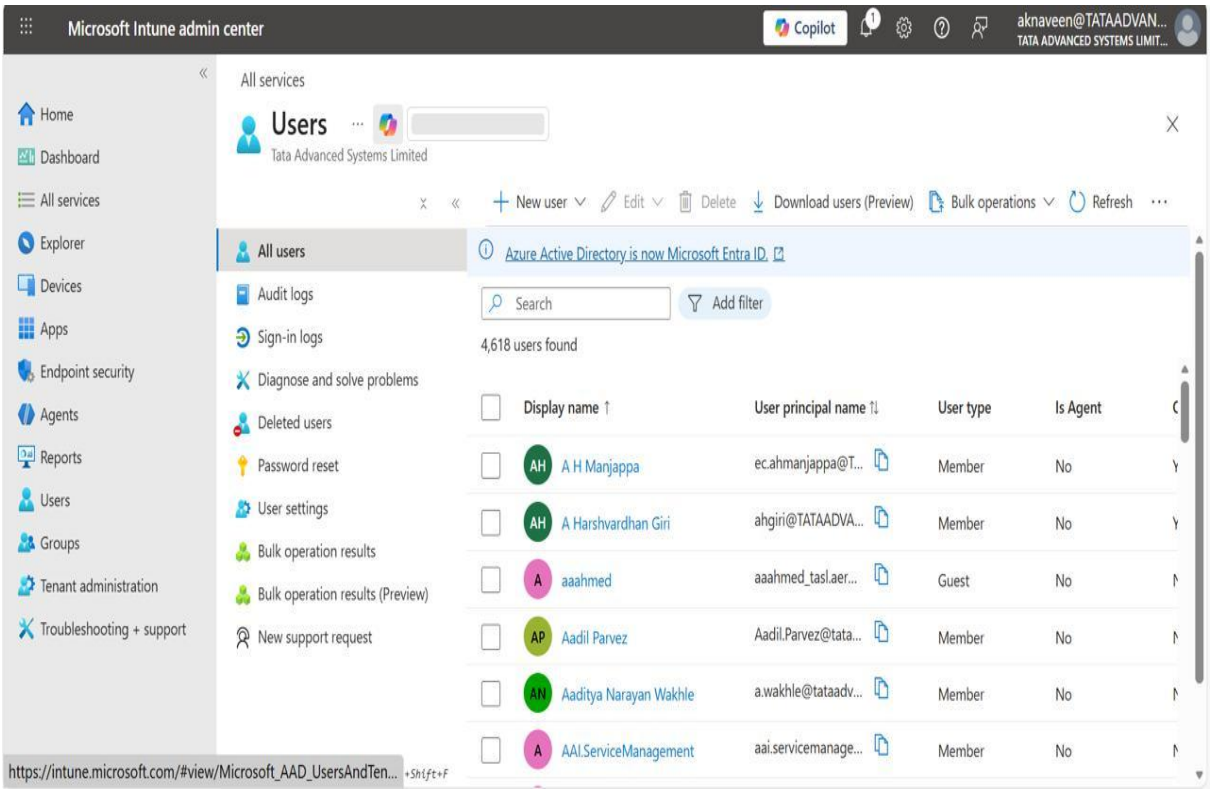
b. Intune Dashboard



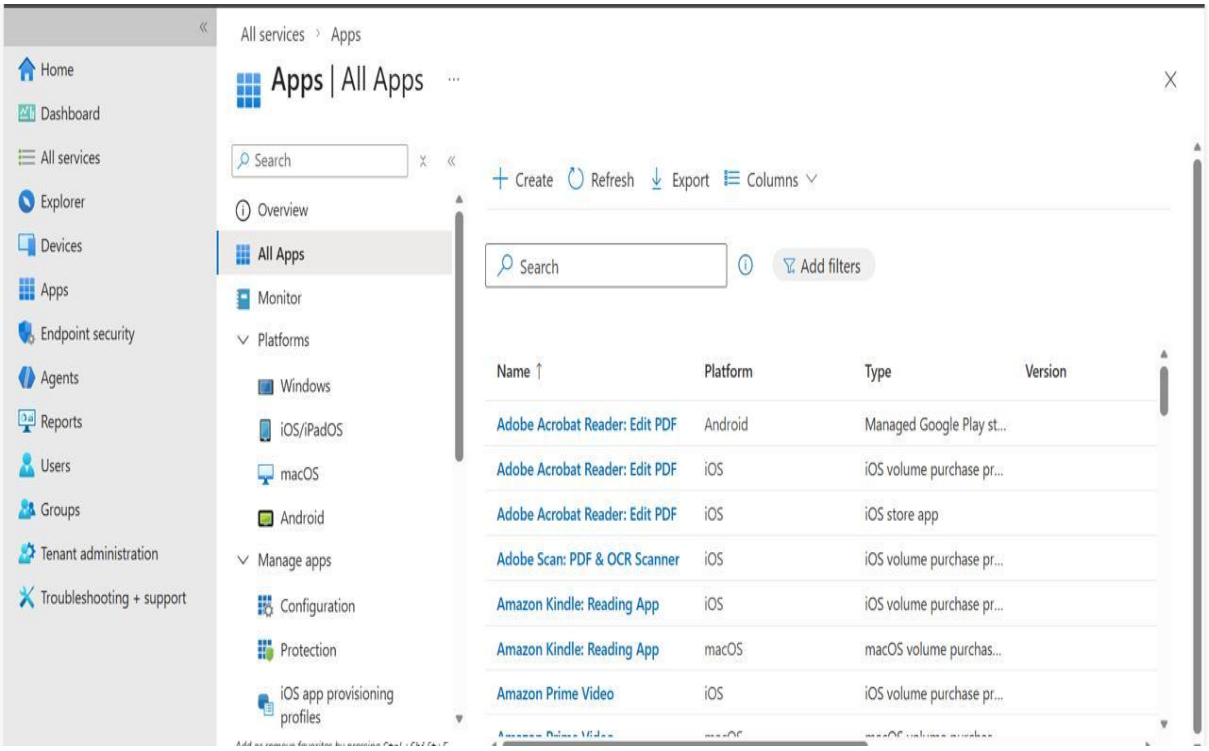
b. IOS Enrollment and Compliance



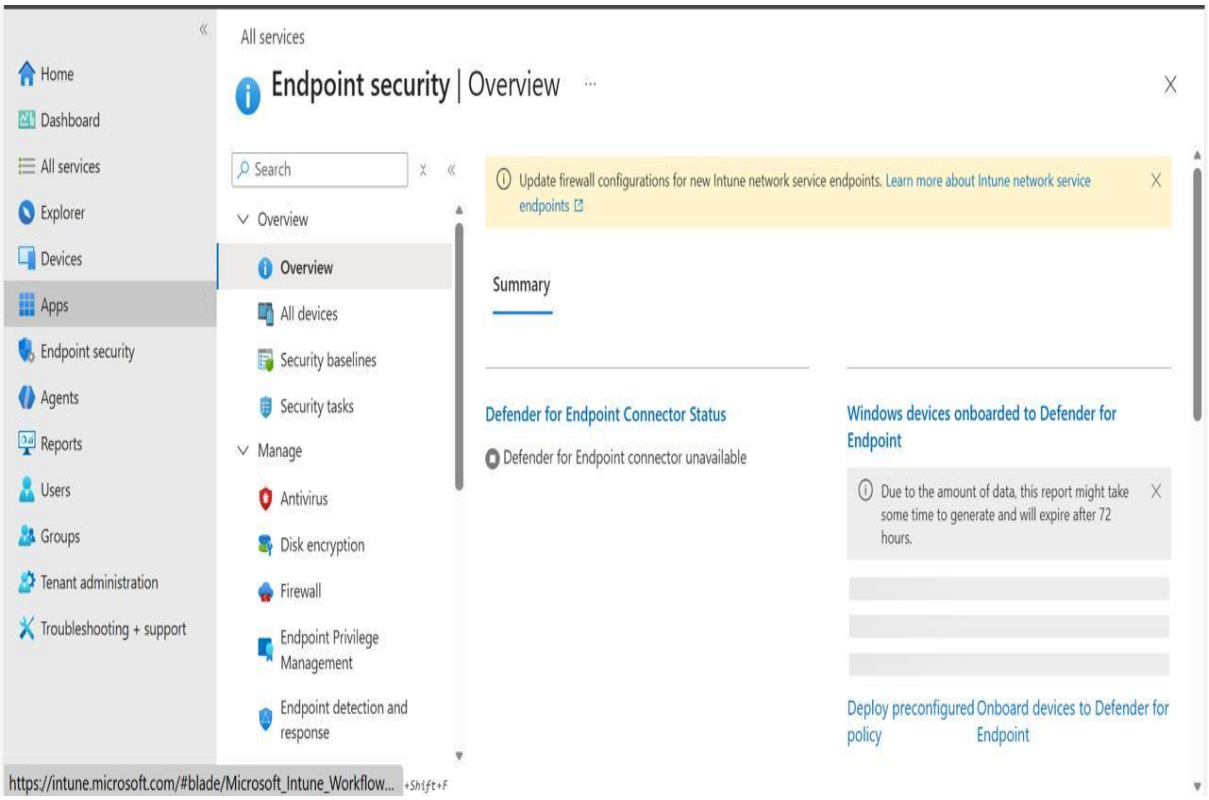
d. Android Enrollment and Compliance



e. Intune All Users and Audit Logs



f. All Apps



g. Endpoint Security

iv. Glossary of Terms

Term	Definition
Azure Function	Serverless compute service used to process WinPak events and trigger Graph API calls
BYOD	Bring Your Own Device – policy allowing personal devices to access corporate resources
Graph API	Microsoft's RESTful API for accessing Azure AD, Intune, and other Microsoft 365 services
Intune	Microsoft's cloud-based enterprise mobility management (EMM) solution
Managed Identity	Azure AD identity automatically managed by Azure; used for secure authentication without secrets
MDM	Mobile Device Management – controls applied to entire devices
MAM	Mobile Application Management – controls applied at the app level (for BYOD)
Restricted Group	Intune group that triggers restrictive policies (camera off, app block)
Serverless	Cloud computing model where infrastructure is fully managed; resources scale automatically
WinPak	Honeywell physical access control system used at TASL for badge-based entry

v. Deployment Checklist

- Graph API permissions granted and admin consented
- Intune Restricted group created with configuration profile assigned
- WinPak webhook URL configured with certificate authentication
- Test devices enrolled and policies validated
- Pilot user list approved and notified
- Monitoring alerts configured
- Fallback manual process documented
- Knowledge transfer session completed with IT team
- Final documentation archived